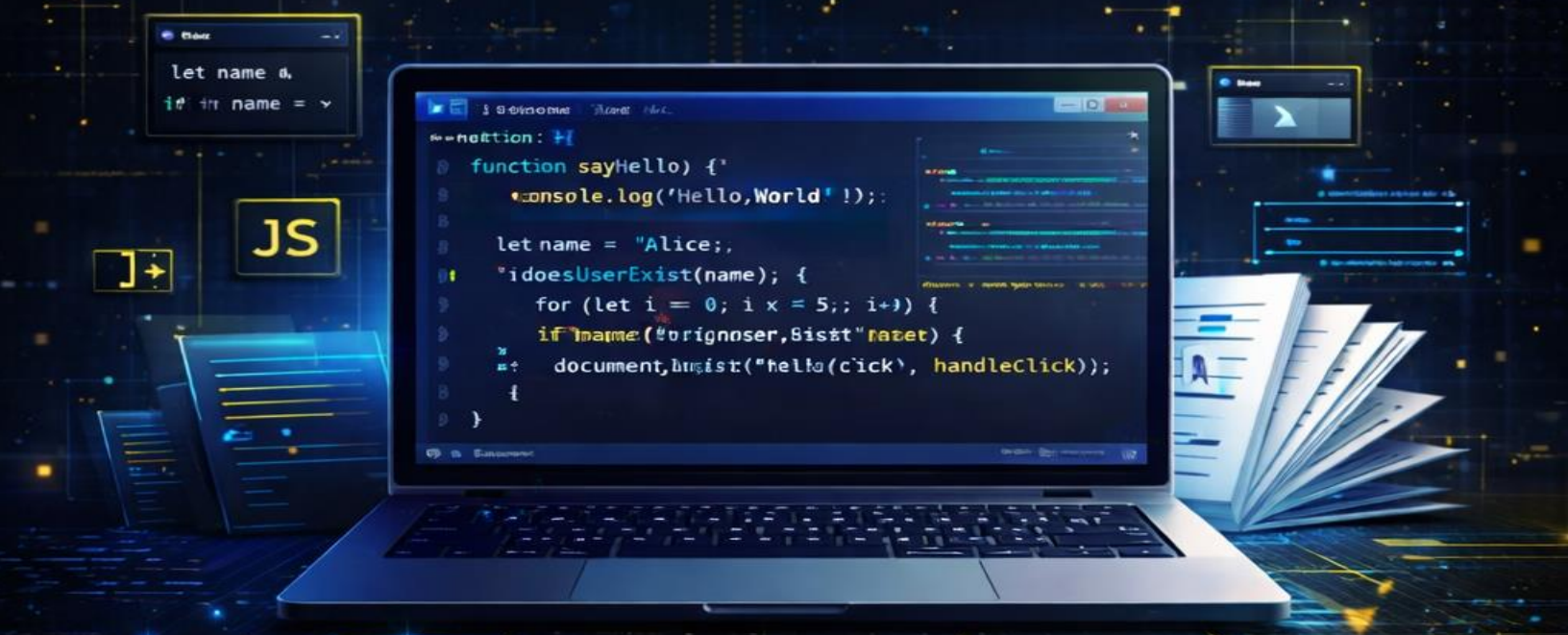
JavaScript

Complete Notes

Learn **JavaScript** from basics to advanced
with real-world examples



Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

Contents

✅ **JavaScript Basics -** 7

1. JavaScript Code Kahan Likhen? (3 Tarike) 7

2. JavaScript Output (Print) Karne Ke Tarike 7

3. JavaScript Basics 7

✅ **JavaScript Lecture 1** 9

1. JavaScript Introduction 9

2. JavaScript Code Ko Run Karne Ke Tarike 10

3. VS Code Setup (Code Editor) 10

4. Our First JavaScript Code 10

5. Variables in JavaScript 11

6. Variable Naming Rules 11

7. let, const aur var 11

8. Data Types in JavaScript 11

JavaScript Lecture 1 Question 12

✅ **Lecture 2: Operators & Conditional Statements** 15

1. Comments in JavaScript 16

2. Operators in JavaScript 16

3. Conditional Statements 17

Questions ANS. 20

Section 1: Basic & Comments (Q1-Q3) 20

Section 2: Arithmetic & Unary Operators (Q4-Q8) 21

Section 3: Comparison & Logical Operators (Q9-Q12) 22

Section 4: Conditional Statements (if-else & else-if) (Q13-Q18) 23

Section 5: Mixed / Advanced (Q19-Q20) 25

✅ **Lecture 3: Loops & Strings** 27

1. Loops Kya Hain? 27

2. for Loop (Sabse Zyada Use Hota Hai) 27

3. while Loop 28

4. Infinite Loop (Warning!) 28

5. for-of Loop (Strings & Arrays ke liye) 28

JavaScript Complete Notes
from basics to advanced
with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

6. for-in Loop (Objects ke liye) 28
7. Strings in JavaScript 29
Template Literals + String Interpolation (Sabse Important) 31
Practice Questions 32
Section 1: Basic Loops (for, while, do-while) 32
Section 2: for-of & for-in Loops 33
Section 3: String Properties & Basic Methods 34
Section 4: Advanced String Methods 35
Section 5: Template Literals & Real Projects 35
✅ JavaScript Chapter 4: Arrays & Array Methods 36
Arrays ka Introduction 36
1. Array Create Karna 37
2. Array Indices (0-based Indexing) 37
3. Looping Over Arrays 38
4. All Important Array Methods 39
Practice Questions 40
Section 1: Basic Array Creation & Indexing (Q1-Q10) 40
Section 2: Mutation Methods (push, pop, shift, unshift, splice) (Q6-Q15) 41
Section 3: Non-Mutation Methods (slice, concat, join, toString) (Q16-Q25) 43
Section 4: Search & Advanced Methods (Q26-Q35) 44
Section 5: Mixed / Real-world (Q36-Q40) 46
Section 6 forEach() 47
Section 7 map() 49
Section 8 filter() 51
Section 9 sort() 52
✅ JavaScript Chapter 5: Functions – 54
JavaScript mein Function Banane ke Sabhi Tarike 57
1. Normal / Traditional Function (Sabse Purana Tarika) 57
2. Function Expression (Anonymous Function) 58
3. Arrow Function (Modern & Most Used Tarika) 58
4. Immediately Invoked Function Expression (IIFE) 59
25 Practical Examples 60

Section 1: Basic Normal Functions (Examples 1-8) 60
Section 2: Arrow Functions (Examples 9-16) 62
Section 3: Mixed Examples (Functions + Previous Concepts) (Examples 17-25) 63
✅ DOM (Document Object Model) 65
1. Introduction 65
2. Kyun important hai DOM? 65
3. Window Object vs Document Object 65
4. DOM Tree & Nodes 66
5. DOM Tree mein Parent, Child & Sibling Relationship 68
6. Selecting / Accessing Elements in DOM 70
7. DOM Properties 70
8. Attributes Handling in DOM 71
9. Styling with JavaScript 72
10. Creating, Inserting, Removing & Replacing Elements in DOM 73
11. DOM Traversal / Navigation 73
12. Event Handling Basics 74
13. Event Object (e / evt) 75
14. DOM Events – Complete Important List 75
15. Advanced Event Handling: Multiple Listeners & removeEventListener() 76
16. Event Delegation 77
DOM Related 20 Practical Questions 78
17. Sabse pehle child element ko hide kar do. 81
✅ Date And Time 81
1. Current Date + Time Kaise Lete Hain? (Sabse Common) 81
2. Important Methods (Sabse zyada use hone wale) 81
Practical Examples (Copy-Paste Kar Ke Try Kar) 82
✅ Timeout 86
1. setTimeout() – Ek Baar Execution (Delay ke Baad) 86
2. clearTimeout() – setTimeout ko Rokna 86
3. setInterval() – Baar-Baar Execution (Har Interval Pe) 87
4. clearInterval() – setInterval ko Rokna 87
Sabka Ek Saath Comparison Table (Sabse Important) 87

10 Practical Questions 88
8. 8 second baad ek welcome message show karo, lekin user chahe toh 4 second mein cancel kar sake. 90
✅ Math Object in JavaScript 93
1. Math Object ke Important Constants (Fixed Values) 93
2. Sabse Important Math Methods (Table) 93
3. Sabse Useful Practical Examples 93
Practical Questions 94
✅ OOP (Object Oriented Programming) 97
OOP Kya Hai? 97
2. Object Kya Hai? (Sabse Basic) 97
3. Class Kya Hai? 97
4. Kyu Use Karte Hain Classes aur Objects? 98
5. Properties Kya Hain? (Detailed Explanation) 98
6. Properties Access Karne Ke Sab Tarike (Yeh Wala Section Pehle Miss Ho Gaya Tha) 98
7. Getters & Setters (Bahut Important) 99
8. new Keyword 100
9. Constructor 100
10. _ (Underscore) Convention 100
11. # Private Fields & Methods 100
12. Inheritance (extends + super) 101
13. Static Methods & Properties 101
14. Prototypal Inheritance 101
15. Constructor Functions (Pre-ES6 Style) 103
16. Method Overriding 103
17. Polymorphism (Ek naam, alag-alag behaviour) 104
18. Abstraction Sirf zaruri details 104
19. 'this' Keyword – Pura Scene (Bahut Important) 105
20. instanceof Operator + Extra Checks 105
Practical Questions 106
✅ Asynchronous JavaScript 112
1. Synchronous vs Asynchronous Programming 112

2. Callback Kya Hai? 113
3. Callback Hell (Pyramid of Doom) 😱 114
4. Promises – Callback Hell ka Solution 114
5. Promise Chaining 115
6. Async / Await (Sabse Best & Modern Way) 🔥 116
7. IIFE (Immediately Invoked Function Expression) + Async/Await 116
8. Final Conclusion (Lecture ka end message) 116
Questions with Answers 116
✅ Fetch API 123
1. API Kya Hai? (Basic Revision) 123
2. Fetch API Kya Hai? 123
3. Fetch se Milne Wala Response Object 124
4. JSON vs XML (Data Format) 124
5. Fetch API with Async/Await (Recommended Way) 124
6. HTTP Methods (GET, POST, etc.) 125
✅ LocalStorage in JavaScript 125
1. LocalStorage Kya Hai? 125
2. Important Methods (Sabse Zyada Use Hone Wale) 126
3. Practical Examples (Copy-Paste Ready) 126
10 Practical Questions 127

JavaScript Complete Notes

learn JavaScript from basics to advanced with real-world examples

Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

✅ JavaScript Basics -

1. JavaScript Code Kahan Likhen? (3 Tarike)

JavaScript code likhne ke 3 common tarike hain:

Tarika	Kaise Likhte Hain	Kab Use Karte Hain
1. HTML file mein	<script> console.log("Hello"); </script>	Chhote tests ke liye
2. External JS File	<script src="script.js"></script> + alag script.js file	Real projects mein (Best Practice)
3. Browser Console	Right Click → Inspect → Console tab	Quick testing & debugging ke liye

Best Practice: Hamesha **External JS file** (script.js) use karo aur usko index.html ke end mein link karo.

2. JavaScript Output (Print) Karne Ke Tarike

JavaScript mein output dikhane ke main tarike:

Method	Kya Karta Hai	Example	Use Case
console.log()	Console mein normal message print karta hai	console.log("Hello World");	Debugging & testing
console.error()	Red color mein error dikhaata hai	console.error("Something went wrong");	Error show karne ke liye
console.warn()	Yellow color mein warning dikhaata hai	console.warn("Warning: Check this");	Warnings dikhane ke liye
console.table()	Array ya object ko table format mein dikhata hai	console.table(["Ram", "Shyam"]);	Data clearly dekhne ke liye
console.clear()	Console ko saaf kar deta hai	console.clear();	Console clean karne ke liye
alert()	Popup box mein message dikhata hai	alert("Hello");	Important message / notification
prompt()	User se input leta hai	let name = prompt("Enter your name");	User input lene ke liye
confirm()	Yes/No (OK/Cancel) popup dikhata hai	let result = confirm("Are you sure?");	Confirmation lene ke liye
document.write()	Page par direct text likhta hai	document.write("Hello World");	**Modern projects mein avoid karo**

Note: alert(), prompt() aur confirm() temporary execution pause kar dete hain.

3. JavaScript Basics

typeof Operator

Variable ka data type batata hai.

JavaScript

THE ARTIFICIAL EDUCATION

```
console.log(typeof 5);            // "number"
console.log(typeof "Hello");      // "string"
console.log(typeof true);         // "boolean"
console.log(typeof []);           // "object"
console.log(typeof undefined);    // "undefined"
```

Comments (Code Explain Karne Ke Liye)

JavaScript

```
// Single Line Comment

/*
   Multi Line Comment
   Yeh comment multiple lines mein likha ja sakta hai
*/
```

Comments code mein run nahi hote. Sirf explanation ke liye hote hain.

Case Sensitivity

JavaScript **case sensitive** hai.

JavaScript

```
let name = "Ram";
let Name = "Shyam";    // Ye dono alag variables hain
```

Semicolon (;)

- Line ko end karta hai.
- Optional hai, lekin **good practice** hai semicolon lagana.

JavaScript

```
let a = 5;
let b = 10;
console.log(a + b);
```

Statements

JavaScript program chhote-chhote statements se banta hai.

JavaScript

```
let a = 5;            // Statement 1
console.log(a);       // Statement 2
```

Keywords (Reserved Words)

Yeh words JavaScript ke liye reserved hain. Inhe variable naam ke roop mein nahi use kar sakte.

Examples: let, const, var, if, else, function, return, for, while, switch

Wrong:

JavaScript

```
let if = 10;    // Error
```

Identifiers (Variable/Function Names)

Rules:

- Letter (a-z ya A-Z) ya _ ya $ se start hona chahiye
- Number se start nahi ho sakta
- Space allowed nahi hai

Correct: userName, _age, $price, totalAmount **Wrong**: 1name, full name

window Object

Browser ka global object hai.

- alert(), prompt(), confirm(), console sab window ke methods hain.
- Hum directly alert("Hello") likh sakte hain, yeh internally window.alert("Hello") hota hai.

✅ JavaScript Lecture 1

1. JavaScript Introduction

JavaScript (JS) duniya ki sabse popular programming languages mein se ek hai। Yeh ek lightweight, interpreted aur high-level programming language hai jo mainly **web development** ke liye bani thi, lekin aaj yeh bahut zyada powerful ho chuki hai।

JavaScript Kya Hai?

JavaScript ek aisi programming language hai jo aapke normal HTML page ko **dynamic aur interactive** bana deti hai। Iske bina aaj ki modern websites adhuri hain।

JavaScript se aap bana sakte ho:

- Interactive Websites
- Single Page Applications (SPA)
- Mobile Apps (React Native)
- Desktop Applications (Electron)
- Backend Servers (Node.js)
- Games
- Real-time applications (Chat apps, Video streaming)

JavaScript ki Short History

- **1995** mein **Brendan Eich** ne Netscape Browser ke liye JavaScript ko banaya tha।
- Shuruaat mein iska naam **Mocha** tha, phir **LiveScript** aur finally **JavaScript** rakha gaya।
- **1997** mein ise **ECMAScript** naam se standardize kiya gaya।
- **2015** mein **ES6 (ECMAScript 6)** aaya, jisme let, const, arrow functions, classes aur modules jaise modern features add hue।

Aaj JavaScript ko **"Web ka Backbone"** bhi kaha jata hai kyunki yeh almost har website mein use hoti hai।

JavaScript Kyun Seekhein?

- Bahut easy aur beginner-friendly hai
- Front-end aur Back-end dono mein kaam aati hai (Full Stack Development)
- Bahut badi community aur job market
- Real projects jaise Netflix, Instagram, WhatsApp Web, Google Maps isi se bane hain
- Ek language seekh kar aap websites, apps aur games sab bana sakte ho

Ek Line Mein JavaScript

JavaScript ek versatile programming language hai jo static web pages ko dynamic, interactive aur intelligent bana deti hai।

2. JavaScript Code Ko Run Karne Ke Tarike

Sabse Simple Tarika: Browser Console mein directly likhna

- Browser kholo (Chrome best hai)
- Right click → Inspect → Console tab
- Ya shortcut: Ctrl + Shift + J

Example:

JavaScript

```
alert("Hello JS");
console.log("Artificial Education");
```

Permanent Code Likhne Ka Tarika: VS Code (code editor) mein .js file banao aur usko HTML file ke saath connect karo.

3. VS Code Setup (Code Editor)

- VS Code ek free aur industry mein sabse popular code editor hai.
- Ek baar install kar lo toh JavaScript, Python, Java, C++ sab ke liye use kar sakte ho.
- File extensions:
  - .js → JavaScript file
  - .html → HTML file

First Project Setup:

1. Ek folder banao (naam: javascript)
2. first.js file banao
3. index.html file banao (Boilerplate ke liye ! press karke Enter)
4. index.html mein last mein ye line add karo:

HTML

```
<script src="first.js"></script>
```

4. Our First JavaScript Code

JavaScript

```
console.log("Artificial Education");
console.log("Welcome to JavaScript!");
```

- console.log() → Console mein message print karta hai.
- Har line ke end mein **semicolon (;)** lagana best practice hai.
- Double quotes ("") ya single quotes ('') dono strings ke liye use kar sakte ho.

Created by Pawan Soni
THE ARTIFICIAL EDUCATION

5. Variables in JavaScript

Variable = Data store karne ka container (jaise dabba)

Example:

- let age = 24;
- let fullName = "Tony Stark";
- let price = 99.99;

Important:

- JavaScript **dynamically typed** hai → Variable ka type batane ki zarurat nahi.
- Variable names meaningful hone chahiye (age, price, fullName).

6. Variable Naming Rules

Rules:

1. Case sensitive (fullName aur FullName alag hain)
2. Sirf letters, digits, _ aur $ allowed
3. Space nahi allowed
4. Number se start nahi ho sakta
5. Reserved keywords (let, const, var, console etc.) use nahi kar sakte

Best Naming (Camel Case):

JavaScript

```
let fullName = "Pawan soni";
let isLoggedIn = true;
let totalPrice = 999;
```

7. let, const aur var

Keyword	Re-declare	Re-assign	Use karna chahiye?
var	Yes	Yes	Kabhi mat use
let	No	Yes	Haan
const	No	No	Sabse best (preferred)

Rules:

- const ko declare karte time value dena zaruri hai.
- let aur const block scoped hote hain.

8. Data Types in JavaScript

Primitive Data Types (7):

- Number → 24, 99.99
- String → "Hello", 'Tony Stark'
- Boolean → true / false
- Undefined → Jab value nahi di hoti
- Null → Khali value (special)
- BigInt → Bahut bade numbers
- Symbol → Unique values

JavaScript
Complete Notes
Learn JavaScript from basics to advanced
with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

Non-Primitive:

- **Object** → Key-value pairs ka collection

Object Example:

JavaScript

```
const student = {
    fullName: "Rahul Kumar",
    age: 21,
    cgpa: 8.5,
    isPass: true
};
```

Access karne ke tarike:

- Dot notation: student.age
- Bracket notation: student["fullName"]

Object mein value change karna (const object ke andar allowed hai):

JavaScript

```
student.age = 22;
student.fullName = "Rahul Sharma";
```

JavaScript Lecture 1 Question

Question 1

Q: Ek variable price banao jiski value 299.99 ho. Phir uski value ko 399.99 kar do aur console mein print karo. Konsa keyword use karoge (let ya const) aur kyun?

Solution:

JavaScript

```
let price = 299.99;
price = 399.99;
console.log(price);   // Output: 399.99
```

Explanation: let use kiya kyunki hum value change (re-assign) kar rahe hain. const se error aata.

Question 2

Q: Niche diye code mein error kya hai aur usko sahi kaise karoge?

JavaScript

```
const userName = "Rahul";
userName = "Aman";
console.log(userName);
```

Solution:

JavaScript

```
let userName = "Rahul";
userName = "Aman";
console.log(userName);   // Output: Aman
```

Explanation: const ki value change nahi ki ja sakti. Isliye let use karna padega.

Question 3

Q: Ek object car banao jisme ye properties hon:

- brand → "Tata"
- model → "Nexon"
- price → 850000
- isElectric → false

Phir is object ko console mein print karo.

Solution:

JavaScript

```
const car = {
    brand: "Tata",
    model: "Nexon",
    price: 850000,
    isElectric: false
};

console.log(car);
```

Question 4

Q: Upar wale car object se brand aur price ko dono tarike se print karo (Dot notation aur Bracket notation).

Solution:

JavaScript

```
console.log(car.brand);          // Dot notation
console.log(car["price"]);       // Bracket notation
```

Question 5

Q: Niche diye variable names ko correct Camel Case mein likho:

a) total amount b) is user logged in c) first name d) final score

Solution:

JavaScript

```
let totalAmount;
let isUserLoggedIn;
let firstName;
let finalScore;
```

Question 6

Q: typeof operator se ye sab check karo aur output likho:

JavaScript

```
let a = 50;
let b = "Hello World";
let c = true;
```

```
let d = null;
let e;

console.log(typeof a);
console.log(typeof b);
console.log(typeof c);
console.log(typeof d);
console.log(typeof e);
```

Solution (Output):

text

```
number
string
boolean
object
undefined
```

Explanation: null ka typeof "object" aata hai (ye JavaScript ka purana bug hai).

Question 7

Q: Ek object mobile banao aur usme ek property price ki value 15000 rakho. Phir us price ko 18000 kar do using JavaScript.

Solution:

JavaScript

```
const mobile = {
    brand: "Samsung",
    price: 15000
};

mobile.price = 18000;
console.log(mobile.price);   // Output: 18000
```

Question 8

Q: Ek variable x banao let se bina value diye. Phir usko console mein print karo. Output kya aayega?

Solution:

JavaScript

```
let x;
console.log(x);            // Output: undefined
console.log(typeof x);     // Output: "undefined"
```

Question 9

Q: Ek const object book banao jisme title aur author ho. Phir author ko change kar do. Kya error aayega?

Solution:

JavaScript

Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

```
const book = {
    title: "Atomic Habits",
    author: "James Clear"
};

book.author = "James Clear New";   // No error
console.log(book.author);
```

Explanation: const object ke andar ki properties (keys) ko change kar sakte hain, sirf pura object nahi badal sakte.

Question 10 (Challenge)

Q: Ek object student banao jisme ye 4 properties hon:

- name → "Priya Sharma"
- age → 20
- marks → 85
- isPass → true

Phir:

1. Is object ko console mein print karo
2. Sirf marks print karo (dono tarike se)
3. marks ko 92 kar do

Solution:

JavaScript

```
const student = {
    name: "Priya Sharma",
    age: 20,
    marks: 85,
    isPass: true
};

console.log(student);                    // 1. Pura object
console.log(student.marks);              // 2. Dot notation
console.log(student["marks"]);           // 2. Bracket notation

student.marks = 92;                      // 3. Update
console.log(student.marks);              // Output: 92
```

✅ Lecture 2: Operators & Conditional Statements

Introduction Is chapter mein hum JavaScript ke **Operators** aur **Conditional Statements** (if-else) ke baare mein seekhenge. Real life mein bhi bahut si conditions hoti hain (jaise age 18+ honi chahiye license ke liye). Wahi logic hum code mein implement karte hain.

1. Comments in JavaScript

Comments code explain karne ke liye likhe jaate hain. Yeh run nahi hote.

Single Line Comment:

JavaScript

```
// Yeh single line comment hai
console.log("Hello World");   // Yeh bhi comment ke saath likh sakte hain
```

Multi Line Comment:

JavaScript

```
/*
   Yeh multi line comment hai
   Isme hum bahut saari lines likh sakte hain
   Code explain karne ke liye useful hai
*/
```

Shortcut (VS Code): Selected code ko comment karne ke liye → Ctrl + /

Best Practice:

- Comments mein clear English likho
- Code ko describe karo (kya kar raha hai)

2. Operators in JavaScript

Operators data pe operations perform karte hain.

A. Arithmetic Operators (Math ke liye)

Operator	Kaam	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
%	Modulus (Remainder)	5 % 2	1
**	Exponentiation (Power)	5 ** 2	25

Increment & Decrement (Unary Operators):

- a++ → Post Increment (pehle use, phir badhao)
- ++a → Pre Increment (pehle badhao, phir use)
- a-- → Post Decrement
- --a → Pre Decrement

B. Assignment Operators

Operator	Example	Meaning
=	a = 5	Assign value
+=	a += 4	a = a + 4
-=	a -= 4	a = a - 4
*=	a *= 4	a = a * 4

JavaScript Complete Notes
Created by Pawan Soni
Learn computer, control your future
THE ARTIFICIAL EDUCATION

/=	a /= 4	a = a / 4
%=	a %= 4	a = a % 4
**=	a **= 4	a = a ** 4

C. Comparison Operators

Operator	Meaning	Example	Result
==	Equal (value only)	5 == "5"	true
===	Strict Equal (value + type)	5 === "5"	false
!=	Not Equal	5 != 3	true
!==	Strict Not Equal	5 !== "5"	true
>	Greater Than	5 > 3	true
<	Less Than	5 < 3	false
>=	Greater Than or Equal	5 >= 5	true
<=	Less Than or Equal	5 <= 5	true

D. Logical Operators

Operator	Meaning	Example	Result
&&	Logical AND (dono true hone chahiye)	(5>3) && (10>8)	true
`		`	Logical OR (koi bhi ek true)
!	Logical NOT (ulita kar deta hai)	!(5>3)	false

Ternary Operator (Short if-else)

JavaScript

```
let result = (age >= 18) ? "Adult" : "Not Adult";
console.log(result);
```

3. Conditional Statements

A. if Statement

JavaScript

```
if (condition) {
    // Code jo tab run hoga jab condition true ho
}
```

B. if-else Statement

JavaScript

```
if (condition) {
    // true case
} else {
    // false case
}
```

C. if-else if-else (Multiple Conditions)

JavaScript

JavaScript Complete Notes
learn JavaScript from basics to advanced with real-world examples
learn computer, control your future
THE ARTIFICIAL EDUCATION

```
if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else if (score >= 70) {
    grade = "C";
} else {
    grade = "F";
}
```

D. Switch Statement (Optional - Homework)

1. Switch Statement Kya Hai?

- Switch ek **conditional statement** hai jo **ek single value** ke multiple possible cases ko check karta hai.
- Yeh if-else if chain ka **short aur clean** alternative hai jab hume **ek hi variable** ke alag-alag values check karne hote hain.
- Real-life example: Traffic light (red → stop, yellow → slow, green → go)

Kab use karein?

- Jab ek variable ke sirf **fixed values** (jaise "red", "green", number 1, 2, 3) check karne hote hain.
- Bahut saare else if conditions hone par switch zyada readable hota hai.

2. Switch Statement ka Syntax

JavaScript

```
switch (expression) {          // expression = wo variable jo check karna hai
    case value1:               // agar expression === value1
        // code here
        break;                 // important! break lagana zaroori hai

    case value2:               // agar expression === value2
        // code here
        break;

    default:                   // agar koi bhi case match na kare
        // default code
}
let fruit = "papaya";

switch (fruit) {
    case "orange":
        console.log("Orange is sweet");
        break;

    case "mango":
```

```
        console.log("Mango is king of fruits");
        break;

    case "papaya":
        console.log("Papaya is good for health");
        break;

    default:
        console.log("Fruit not found in our list");
}

let score = 85;
let grade;

switch (true) {                    // special trick: switch(true) use kar sakte
hain
    case (score >= 90):
        grade = "A";
        break;
    case (score >= 80):
        grade = "B";
        break;
    case (score >= 70):
        grade = "C";
        break;
    case (score >= 60):
        grade = "D";
        break;
    default:
        grade = "F";
}

console.log("Your grade is " + grade);   // Your grade is B
```

3. Simple Example (Lecture wala Fruit Example)

JavaScript

```
// Output: Papaya is good for health
```

Agar break na lagayein toh kya hoga? Sab cases se neeche wale bhi execute ho jayenge (fall-through behaviour).

THE ARTIFICIAL EDUCATION

4. Real-life Practical Example (Grade Calculator with Switch)

JavaScript

5. Switch vs if-else (Comparison Table)

Feature	if-else	switch
Best for	Multiple different conditions	Ek variable ke multiple fixed values
Readability	Bahut saare else-if hone par messy	Clean aur short
Performance	Thoda slow (sab check karta hai)	Fast (direct jump)
Default case	else	default
Range check	Easy (>=, <=)	Thoda tricky (switch(true) trick)

Rule of Thumb:

- Agar sirf **ek variable** ke different fixed values check karne hain → **Switch** use karo.
- Agar complex conditions (ranges, &&, ||) hain → **if-else** better.

6. Important Points

- break lagana **zaroori** hai warna neeche wale cases bhi chal jayenge.
- default case optional hai lekin best practice hai.
- Switch mein string, number, boolean sab use kar sakte hain.
- Case sensitive hota hai ("Papaya" aur "papaya" alag hain).

Questions ANS.

Section 1: Basic & Comments (Q1-Q3)

Q1. JavaScript mein single line aur multi-line comment ka syntax likho aur ek example do.

Ans:

JavaScript

```
// Single Line Comment
console.log("Hello");   // Yeh bhi single line comment hai

/*
   Multi Line Comment
   Isme multiple lines likh sakte hain
   Code explain karne ke liye best
*/
```

Q2. Ek program likho jisme console.log("Artificial Education") ke upar ek meaningful comment ho.

Ans:

JavaScript

```
// Yeh line console pe "Artificial Education" print karti hai
console.log("Artificial Education");
```

Q3. Comment out karne ka shortcut kya hai VS Code mein?

Ans: Selected code ko comment/uncomment karne ke liye → **Ctrl + /**

Section 2: Arithmetic & Unary Operators (Q4-Q8)

Q4. let a = 10; let b = 3; Inke liye saare arithmetic operators ka result print karo.

Ans:

JavaScript

```
let a = 10;
let b = 3;
console.log("a + b =", a + b);     // 13
console.log("a - b =", a - b);     // 7
console.log("a * b =", a * b);     // 30
console.log("a / b =", a / b);     // 3.333...
console.log("a % b =", a % b);     // 1
console.log("a ** b =", a ** b);   // 1000
```

Q5. Pre-increment aur Post-increment ka difference samjhao ek example ke saath.

Ans:

JavaScript

```
let a = 5;
console.log(++a);   // Pre → pehle +1 phir print → 6
console.log(a);     // 6

let b = 5;
console.log(b++);   // Post → pehle print phir +1 → 5
console.log(b);     // 6
```

Q6. let x = 8; x += 5, x -= 3, x *= 2, x %= 7 — har step ke baad value print karo.

Ans:

JavaScript

```
let x = 8;
x += 5;   console.log(x);   // 13
x -= 3;   console.log(x);   // 10
x *= 2;   console.log(x);   // 20
x %= 7;   console.log(x);   // 6
```

Q7. Ek number odd hai ya even check karne ka code likho (modulus use karke).

Ans:

JavaScript

```
let num = 17;
if (num % 2 === 0) {
```

```
    console.log(num + " is Even");
} else {
    console.log(num + " is Odd");
}
// Output: 17 is Odd
```

Q8. let score = 85; Ternary operator use karke print karo "Pass" ya "Fail" (passing marks 40).

Ans:

JavaScript

```
let score = 85;
let result = (score >= 40) ? "Pass" : "Fail";
console.log(result);   // Pass
```

Section 3: Comparison & Logical Operators (Q9-Q12)

Q9. == aur === mein kya farak hai? Ek example do jisme dono alag result de.

Ans:

JavaScript

```
console.log(5 == "5");     // true  (sirf value check)
console.log(5 === "5");    // false (value + type dono check)
```

Q10. Logical AND (&&) aur OR (||) ka use karke ek condition banao: Age 18+ ho **aur** Indian citizen ho to "You can vote" print ho.

Ans:

JavaScript

```
let age = 20;
let isIndian = true;

if (age >= 18 && isIndian) {
    console.log("You can vote");
} else {
    console.log("You cannot vote");
}
```

Q11. ! (NOT) operator ka example do.

Ans:

JavaScript

```
let isRaining = false;
console.log(!isRaining);   // true
```

Q12. Ek program banao jo check kare ki number 5 ka multiple hai ya nahi (prompt use karke).

Ans:

JavaScript

```
let num = parseInt(prompt("Enter a number"));
if (num % 5 === 0) {
    console.log(num + " is multiple of 5");
} else {
    console.log(num + " is NOT multiple of 5");
}
```

Complete Notes

Section 4: Conditional Statements (if-else & else-if) (Q13-Q18)

Q13. Ek program banao jo user se age le aur bataaye:

- 18+ → "Adult"
- 13-17 → "Teenager"
- 0-12 → "Child"

Learn JavaScript from basics to advanced
with real-world examples

Ans:

JavaScript

```
let age = parseInt(prompt("Enter your age"));
if (age >= 18) {
    console.log("Adult");
} else if (age >= 13) {
    console.log("Teenager");
} else {
    console.log("Child");
}
```

Q14. Grade calculator (Lecture mein diya gaya): 90-100 → A 80-89 → B 70-79 → C 60-69 → D 0-59 → F

Ans:

JavaScript

```
let score = 85;
let grade;

if (score >= 90) grade = "A";
else if (score >= 80) grade = "B";
else if (score >= 70) grade = "C";
else if (score >= 60) grade = "D";
else grade = "F";

console.log("Your grade is " + grade);
```

Q15. Dark/Light mode wala example: Mode = "dark" ho to background black, warna white.

Ans:

JavaScript

```
let mode = "dark";
let color;
```

```
if (mode === "dark") {
    color = "black";
} else {
    color = "white";
}
console.log("Background color is " + color);
```

Q16. User se ek number lo. Agar number 100 se bada ho to "Big Number", 50-100 ke beech "Medium", warna "Small Number".

Ans:

JavaScript

```
let num = parseInt(prompt("Enter number"));
if (num > 100) {
    console.log("Big Number");
} else if (num >= 50) {
    console.log("Medium Number");
} else {
    console.log("Small Number");
}
```

Q17. Ternary operator use karke Q15 ko short mein likho.

Ans:

JavaScript

```
let mode = "dark";
let color = (mode === "dark") ? "black" : "white";
console.log(color);
```

Q18. Ek program banao jo check kare ki koi number positive, negative ya zero hai.

Ans:

JavaScript

```
let num = parseInt(prompt("Enter a number"));
if (num > 0) {
    console.log("Positive");
} else if (num < 0) {
    console.log("Negative");
} else {
    console.log("Zero");
}
```

Section 5: Mixed / Advanced (Q19-Q20)

Q19. Assignment operator ka use karke x = 10 ko 5 se multiply karo aur phir 3 se divide karo (ek line mein).

Ans:

JavaScript

```
let x = 10;
x *= 5;   // 50
x /= 3;   // 16.666...
console.log(x);
```

Q20. Ek program banao jisme user se 3 numbers le aur sabse bada number print karo (if-else-if use karke).

Ans:

JavaScript

```
let a = 45, b = 78, c = 23;
let largest;

if (a >= b && a >= c) largest = a;
else if (b >= a && b >= c) largest = b;
else largest = c;

console.log("Largest number is " + largest);   // 78
```

Q21. User se ek number 1 se 7 tak lo (1 = Monday, 2 = Tuesday ... 7 = Sunday). Switch use karke din ka naam print karo. Agar 1-7 ke bahar number ho to "Invalid day" print karo.

Solution:

JavaScript

```
let day = parseInt(prompt("Enter number between 1 to 7"));

switch (day) {
    case 1:
        console.log("Monday");
        break;
    case 2:
        console.log("Tuesday");
        break;
    case 3:
        console.log("Wednesday");
        break;
    case 4:
        console.log("Thursday");
        break;
```

```
    case 5:
        console.log("Friday");
        break;
    case 6:
        console.log("Saturday");
        break;
    case 7:
        console.log("Sunday");
        break;
    default:
        console.log("Invalid day! Please enter 1-7");
}
```


Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

✅ Lecture 3: Loops & Strings

Lecture Overview Is chapter mein hum **Loops** (code ko repeat karne ka tarika) aur **Strings** (text handle karne ka complete concept) cover kar rahe hain. Yeh dono JavaScript ke bahut important pillars hain.

1. Loops Kya Hain?

Loops ka kaam hai **ek hi code ko baar-baar repeat** karna. Agar hume koi task 5 baar, 100 baar ya 1 lakh baar karna ho to manually copy-paste nahi karna padta – loop automatically kar deta hai.

Real-life Example: Spotify pe gaana loop pe chalta hai, ya YouTube pe video repeat hota hai.

Types of Loops (sabse important):

Loop Type	Kab Use Karein	Syntax Highlight
for loop	Jab exact count pata ho (sabse common)	for (init; condition; update)
while loop	Jab condition pehle check karni ho	while (condition)
do-while loop	Kam se kam ek baar toh chalna hi chahiye	do { } while (condition)
for-of loop	Strings aur Arrays ke har element pe	for (let val of string)
for-in loop	Objects ki keys pe	for (let key in object)

2. for Loop (Sabse Zyada Use Hota Hai)

Syntax:

JavaScript

```
for (initialization; stopping condition; updation) {
    // code jo repeat hoga
}
```

Example 1: "Artificial Education" 5 baar print karo

JavaScript

```
for (let count = 1; count <= 5; count++) {
    console.log("Artificial Education");
}
```

Flow Step-by-Step:

1. **Initialization** → sirf ek baar hota hai (count = 1)
2. **Condition check** → true hai to loop andar jaata hai
3. **Code execute** → andar ka code chalta hai
4. **Updation** → count++ hota hai
5. Phir condition check → false hone tak repeat

Example 2: 1 se 5 tak numbers print karo

JavaScript

```
for (let i = 1; i <= 5; i++) {
    console.log("i =", i);
}
```

Important Point:

- Loop ke andar banaya variable (let i) sirf usi loop tak limited hota hai (block scope).
- var use karoge to bahar bhi accessible hoga (isliye let hi use karo).

3. while Loop

Condition pehle check hoti hai.

Syntax:

JavaScript

```
let i = 1;          // initialization bahar
while (i <= 5) {    // condition
    console.log("i =", i);
    i++;            // updation andar
}
```

Do-while Loop (kam se kam ek baar toh chalega)

JavaScript

```
let i = 20;
do {
    console.log("i =", i);   // pehli baar toh chalega hi
    i++;
} while (i <= 10);
```

4. Infinite Loop (Warning!)

Jab stopping condition kabhi false hi nahi hoti:

JavaScript

```
for (let i = 1; i >= 0; i++) {   // yeh kabhi rukega nahi
    console.log(i);
}
```

Nuksan: Browser hang ho sakta hai, memory full ho sakti hai. Hamesha condition sahi rakho.

5. for-of Loop (Strings & Arrays ke liye)

Har character/element pe loop lagata hai.

JavaScript

```
let str = "ArtificialEducation";

for (let char of str) {
    console.log("char =", char);
}
```

6. for-in Loop (Objects ke liye)

Object ki saari **keys** deta hai.

JavaScript

JavaScript Complete Notes
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

```
let student = {
    name: "Rahul Kumar",
    age: 20,
    cgpa: 7.5,
    isPass: true
};

for (let key in student) {
    console.log("key =", key, "value =", student[key]);
}
```

7. Strings in JavaScript

Strings ka Introduction

String JavaScript mein **text** ko store karne ka tarika hai. Har string characters (letters, numbers, symbols, spaces) ka collection hoti hai.

Key Points:

- String **immutable** hoti hai → original string kabhi change nahi hoti. Koi bhi method nayi string return karta hai.
- String ko 3 tarike se create kar sakte hain:

JavaScript

```
let str1 = "Artificial Education";     // double quotes
let str2 = 'Hello JS';          // single quotes
let str3 = `Template Literal`; // backticks (sabse powerful)
```

- Template literals (backticks) mein hum variables aur expressions directly embed kar sakte hain → **String Interpolation** kehte hain.

JavaScript

```
let name = "Rahul";
console.log(`Hello ${name}, your score is ${85 + 5}`);
```

String kaam kyun aati hai?

- User input (name, password, message)
- Text display aur manipulation
- Username generation, formatting, search, replace etc.

All Important String Methods (Complete List)

Method	Kaam (What it does)	Syntax & Example	Output Example
.length	String ki total characters ki length	"ArtificialEducation".length	20
.toUpperCase()	Saari string uppercase mein	"hello".toUpperCase()	"HELLO"

Learn JavaScript from basics to advanced with real-world examples
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

.toLowerCase()	Saari string lowercase mein	"HELLO".toLowerCase()	"hello"
.trim()	Start aur end ki extra spaces hata deta hai	" hello ".trim()	"hello"
.trimStart()	Sirf starting spaces hata deta hai	" hello".trimStart()	"hello"
.trimEnd()	Sirf ending spaces hata deta hai	"hello ".trimEnd()	"hello"
.slice(start, end)	String ka tukda (end non-inclusive)	"ArtificialEducation".slice(0, 4)	"Arti"
.substring(start, end)	Slice jaisa, lekin negative index nahi leta	"ArtificialEducation".substring(4, 8)	"fici"
.charAt(index)	Kisi index ka single character	"Apna".charAt(2)	"n"
.indexOf(search)	Pehli baar matching substring ki position (nahi mila to -1)	"ArtificialEducation".indexOf("ficial")	4
.lastIndexOf(search)	Aakhri baar matching substring ki position	"abcabc".lastIndexOf("a")	3
.includes(search)	String mein wo substring hai ya nahi (true/false)	"ArtificialEducation".includes(" Education")	true
.startsWith(search)	String is search se shuru hoti hai ya nahi	"ArtificialEducation".startsWith("Arti")	true

JavaScript Complete Notes
Learn JavaScript from basics to advanced with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

.endsWith(search)	String is search pe khatam hoti hai ya nahi	"ArtificialEducation".endsWith("n")	true
.replace(old, new)	Pehli matching value ko replace karta hai	"Artificial Education".replace("Artificial ", "My")	"My Education "
.replaceAll(old, new)	Saari matching values ko replace karta hai	"aaa".replaceAll("a", "X")	"XXX"
.concat(str2)	Do ya zyada strings ko jodta hai	"Hello".concat(" ", "World")	"Hello World"
+ operator	Sabse simple concatenation	"Hello" + " " + "World"	"Hello World"
.split(separator)	String ko array mein tod deta hai	"Artificial Education".split(" ")	["Artificial ", " Education "]
.repeat(count)	String ko kitni baar repeat karta hai	"JS ".repeat(3)	"JS JS JS "
.padStart(length, char)	String ko left side se characters add karta hai	"5".padStart(3, "0")	"005"
.padEnd(length, char)	String ko right side se characters add karta hai	"5".padEnd(3, "0")	"500"

Template Literals + String Interpolation (Sabse Important)

JavaScript

```
let item = "Laptop";
let price = 65000;
```

JavaScript Complete Notes
Learn JavaScript from basics to advanced with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

```
console.log(`The cost of ${item} is ₹${price}`);
// Output: The cost of Laptop is ₹65000
```

Escape Characters (Special)

- \n → New line
- \t → Tab space
- \' → Single quote
- \" → Double quote

Example:

JavaScript

```
console.log("Line 1\nLine 2\tTabbed");
```

Practice Questions

Section 1: Basic Loops (for, while, do-while)

Q1. for loop use karke 1 se 10 tak saare numbers print karo.

Ans:

JavaScript

```
for (let i = 1; i <= 10; i++) {
    console.log(i);
}
```

Explanation: Simple for loop. Initialization i=1, condition i<=10, updation i++.

Q2. while loop use karke 5 se 15 tak saare odd numbers print karo.

Ans:

JavaScript

```
let i = 5;
while (i <= 15) {
    if (i % 2 !== 0) {
        console.log(i);
    }
    i++;
}
```

Q3. do-while loop use karke user se number input lo aur tab tak repeat karo jab tak user 0 na enter kare.

Ans:

JavaScript

```
let num;
do {
    num = parseInt(prompt("Enter a number (0 to stop)"));
    console.log("You entered:", num);
} while (num !== 0);
console.log("Game Over!");
```

Q4. 1 se 50 tak saare even numbers ka sum calculate karo aur print karo (for loop use karke).

Ans:

JavaScript

```
let sum = 0;
for (let i = 2; i <= 50; i += 2) {
    sum += i;
}
console.log("Sum of even numbers 1-50 =", sum);  // Output: 650
```

Q5. Ek program banao jo 10 se 1 tak numbers countdown kare (reverse for loop).

Ans:

JavaScript

```
for (let i = 10; i >= 1; i--) {
    console.log(i);
}
```

Q6. Infinite loop ka example likho aur explain karo ki isse kyun bachna chahiye.

Ans:

JavaScript

```
// Infinite loop (browser hang kar sakta hai)
for (let i = 1; i >= 0; i++) {
    console.log(i);
}
```

Explanation: Condition i >= 0 kabhi false nahi hogi. Memory full ho sakti hai.

Section 2: for-of & for-in Loops

Q7. Ek string "JavaScript" ke har character ko for-of loop se print karo.

Ans:

JavaScript

```
let str = "JavaScript";
for (let char of str) {
    console.log(char);
}
```

Q8. Ek object banao aur for-in loop use karke saari keys aur values print karo.

Ans:

Created by Pawan Soni

JavaScript

```
let student = {
    name: "Aman",
    age: 21,
    course: "BCA"
};

for (let key in student) {
    console.log(key + " =", student[key]);
}
```

Q9. Ek string ke har character ko uppercase mein print karo using for-of.

Ans:

JavaScript

```
let str = "hello";
for (let char of str) {
    console.log(char.toUpperCase());
}
```

Q10. Ek object ke saare values ka sum calculate karo (sirf numbers wale) using for-in.

Ans:

JavaScript

```
let obj = { a: 10, b: 20, c: 30, name: "test" };
let sum = 0;
for (let key in obj) {
    if (typeof obj[key] === "number") {
        sum += obj[key];
    }
}
console.log("Sum =", sum);  // 60
```

Section 3: String Properties & Basic Methods

Q11. Ek string " Artificial Education " ki length print karo aur .trim() use karke spaces hatao.

Ans:

JavaScript

```
let str = "  Artificial Education  ";
console.log("Original length =", str.length);     // 24
console.log("After trim =", str.trim().length);   // 20
```

Q12. Ek string ko uppercase aur phir lowercase mein convert karo.

Ans:

JavaScript

```
let str = "JavaScript";
console.log(str.toUpperCase());   // JAVASCRIPT
console.log(str.toLowerCase());   // javascript
```

Q13. String "ArtificialEducation" ka 4th character (0-based index) print karo using .charAt().

Ans:

JavaScript

```
let str = "ArtificialEducation";
console.log(str.charAt(4));   // f
```

Q14. Do strings "Hello" aur "World" ko + operator aur .concat() dono se join karo.

Ans:

JavaScript

```
let a = "Hello";
let b = "World";
```

```
console.log(a + " " + b);           // Hello World
console.log(a.concat(" ", b));      // Hello World
```

Section 4: Advanced String Methods

Q15. String "I love JavaScript" se "JavaScript" word slice karke nikaalo.

Ans:

JavaScript

```
let str = "I love JavaScript";
console.log(str.slice(7, 17));   // JavaScript
```

Q16. String "Artificial Education" mein "College" ko "University" se replace karo.

Ans:

JavaScript

```
let str = "Artificial Education";
console.log(str.replace("Education ", "University"));  // Artificial University
```

Q17. Ek program banao jo string ke start aur end ki spaces ko trim kare aur length print kare.

Ans:

JavaScript

```
let str = "   Hello World   ";
console.log("Before trim:", str.length);        // 17
console.log("After trim:", str.trim().length);  // 11
```

Q18. String "abcabcabc" mein saare "a" ko "X" se replace karo (.replaceAll use karke).

Ans:

JavaScript

```
let str = "abcabcabc";
console.log(str.replaceAll("a", "X"));   // XbcXbcXbc
```

Section 5: Template Literals & Real Projects

Q19. Template literal use karke dynamic sentence banao: "The price of [item] is ₹[price]"

Ans:

JavaScript

```
let item = "Laptop";
let price = 65000;
console.log(`The price of ${item} is ₹${price}`);
```

Q20. User se full name lo aur username generate karo: @fullName + length (jaise Lecture mein tha).

Ans:

JavaScript

```
let fullName = prompt("Enter your full name (no spaces)");
let username = `@${fullName}${fullName.length}`;
console.log("Your username is", username);
```

Q21. Escape characters use karke multi-line output print karo.

Ans:

THE ARTIFICIAL EDUCATION

JavaScript

```
console.log("Hello\nWorld\nWelcome to JS");
```

Q22. Ek program banao jo user se ek sentence le aur uski length + first aur last character print kare (template literal use karke).

Ans:

JavaScript

```
let sentence = prompt("Enter any sentence");
console.log(`Length = ${sentence.length}, First = ${sentence[0]}, Last =
${sentence[sentence.length-1]}`);
```

Q23. 0 se 100 tak saare even numbers print karo (for loop + if use karke).

Solution:

JavaScript

```
for (let num = 0; num <= 100; num++) {
    if (num % 2 === 0) {
        console.log(num);
    }
}
```

Q24. Ek game banao: Random game number (jaise 25) fix karo. User se prompt leke guess karwao. Jab tak sahi na guess kare, "Wrong number, guess again" dikhao. Sahi guess karte hi "Congratulations, you entered the right number" print karo.

Solution:

JavaScript

```
let gameNum = 25;
let userNum = prompt("Guess the game number");

while (userNum != gameNum) {
    userNum = prompt("Wrong number! Guess again");
}

console.log("Congratulations, you entered the right number");
```

JavaScript with real-world examples
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

✅ JavaScript Chapter 4: Arrays & Array Methods

Arrays ka Introduction

Array JavaScript mein **ek linear collection** hai jisme hum multiple values (same ya different type ki) ko ek hi variable ke andar store kar sakte hain |

Real-life Example:

- Ek class ke saare students ke marks
- Ek shopping website ke products ki list
- Ek game ke high scores

Array kyun use karte hain?

- Multiple related values ko ek naam ke andar rakh sakte hain
- Variables banane ki zarurat nahi padti (jaise student1, student2...)
- Loop laga kar har item pe easily kaam kar sakte hain
- Indexing ke through kisi bhi position ki value turant access kar sakte hain

Important Difference (String vs Array)

- String → **Immutable** (original string kabhi change nahi hoti)
- Array → **Mutable** (original array ko directly change kar sakte hain)

1. Array Create Karna

Syntax:

JavaScript

```
let arrayName = [item1, item2, item3, ...];
```

Examples:

JavaScript

```
// Numbers ka array
let marks = [97, 82, 75, 64, 36];

// Strings ka array
let heroes = ["Thor", "Spider-Man", "Iron Man", "Hulk"];

// Mixed type (not recommended for beginners)
let mixed = [1, "hello", true, 3.14];
```

Array ki length check karne ka tarika:

JavaScript

```
console.log(marks.length);  // 5
```

2. Array Indices (0-based Indexing)

- Array mein har item ki position **0 se start** hoti hai |
- Last index = array.length - 1

Example:

JavaScript

```
let marks = [97, 82, 75, 64, 36];

console.log(marks[0]);  // 97
console.log(marks[1]);  // 82
console.log(marks[4]);  // 36
console.log(marks[5]);  // undefined (kyunki index 5 exist nahi karta)
```

Value change karna (Mutability)

JavaScript

JavaScript Complete Notes
Learn JavaScript from basics to advanced with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

```
marks[2] = 80;        // 75 ki jagah 80 ho gaya
console.log(marks);     // [97, 82, 80, 64, 36]
```

3. Looping Over Arrays

Best ways:

A. for Loop (Sabse common)

JavaScript

```
let heroes = ["Thor", "Spider-Man", "Iron Man"];

for (let i = 0; i < heroes.length; i++) {
  console.log(heroes[i]);
}
```

B. for-of Loop (Sabse clean for arrays)

JavaScript

```
for (let hero of heroes) {
  console.log(hero);
}
```

JavaScript
Complete Notes
Created by **Pawan Soni**
learn computer, control your future
THE ARTIFICIAL EDUCATION

4. All Important Array Methods

Method	Kaam (What it does)	Syntax	Changes Original Array?	Example
.length	Array ki length (number of items) deta hai	arr.length	No	[1,2,3].length → 3
.push()	End mein ek ya multiple items add karta hai	arr.push(item1, item2)	**Yes**	arr.push(99)
.pop()	End se last item remove karta hai aur return karta hai	arr.pop()	**Yes**	Removes last item
.shift()	Start se first item remove karta hai	arr.shift()	**Yes**	Removes first item
.unshift()	Start mein ek ya multiple items add karta hai	arr.unshift(item1, item2)	**Yes**	arr.unshift(0)
.splice()	Add, remove, replace sab kuch karta hai (sabse powerful)	arr.splice(start, deleteCount, newItems)	**Yes**	arr.splice(2, 1, 99)
.slice()	Array ka tukda (slice) nikaalta hai	arr.slice(start, end)	No	arr.slice(1, 4)
.concat()	Multiple arrays ko join karta hai	arr1.concat(arr2, arr3)	No	Returns new array
.toString()	Array ko comma-separated string mein convert karta hai	arr.toString()	No	"1,2,3"
.join(separator)	Array ko string mein join karta hai (custom separator)	arr.join("-")	No	"1-2-3"
.indexOf(item)	Item ki first position deta hai (-1 if not found)	arr.indexOf("Thor")	No	-1 if not found
.lastIndexOf(item)	Item ki last position deta hai	arr.lastIndexOf("a")	No	Last occurrence
.includes(item)	Item array mein hai ya nahi (true/false)	arr.includes("Hulk")	No	true / false
.find(callback)	Pehla item jo condition satisfy kare, return karta hai	arr.find(item => item > 50)	No	First matching item
.findIndex(callback)	Pehla item jo condition satisfy kare, uska index deta hai	arr.findIndex(item => item > 50)	No	Index of first match

JavaScript Complete Notes
Learn JavaScript from basics to advanced with real-world examples
Created by Pawan Soni
Learn computer, control your future
THE ARTIFICIAL EDUCATION

.forEach(callback)	Har item pe callback function run karta hai	arr.forEach(item => console.log(item))	No	Loops without return
.map(callback)	Har item ko transform karke **naya array** return karta hai	arr.map(item => item * 2)	No	New transformed array
.filter(callback)	Condition true hone wale items ka **naya array** return karta hai	arr.filter(item => item > 50)	No	New filtered array
.reduce(callback, initialValue)	Sab items ko ek single value mein reduce karta hai	arr.reduce((acc, curr) => acc + curr, 0)	No	Sum, product etc.
.some(callback)	Koi bhi ek item condition satisfy karta hai? (true/false)	arr.some(item => item > 100)	No	true if any match
.every(callback)	Saare items condition satisfy karte hain? (true/false)	arr.every(item => item > 0)	No	true only if all match
.sort()	Array ko alphabetically ya numerically sort karta hai	arr.sort() or arr.sort((a,b)=>a-b)	**Yes**	In-place sorting
.reverse()	Array ko reverse karta hai	arr.reverse()	**Yes**	In-place reverse
.fill(value, start, end)	Array ke sabhi elements ko ek value se fill karta hai	arr.fill(0)	**Yes**	Fills with same value
.flat(depth)	Nested arrays ko flat (ek level) karta hai	arr.flat(2)	No	Removes nesting
.flatMap(callback)	map() + flat() ek saath	arr.flatMap(x => [x, x*2])	No	Map then flatten
.copyWithin(target, start, end)	Array ke andar hi elements copy karta hai	arr.copyWithin(0, 3)	**Yes**	Advanced copy

Practice Questions

Section 1: Basic Array Creation & Indexing (Q1-Q10)

Q1. Ek array marks banao jisme 5 students ke marks hon: [85, 97, 44, 37, 76]. Is array ki length print karo.

Ans:

JavaScript

```
let marks = [85, 97, 44, 37, 76];
console.log(marks.length);   // Output: 5
```

Q2. Upar wale array mein index 2 ki value ko 80 se replace karo aur pura array print karo.

Ans:

JavaScript

JavaScript Complete Notes
Learn JavaScript from basics to advanced with real-world examples
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

```
let marks = [85, 97, 44, 37, 76];
marks[2] = 80;
console.log(marks);   // [85, 97, 80, 37, 76]
```

Q3. Ek array fruits banao: ["Apple", "Banana", "Mango", "Orange"]. Isme se index 1 aur index 3 ke fruits print karo.

Ans:

JavaScript

```
let fruits = ["Apple", "Banana", "Mango", "Orange"];
console.log(fruits[1]);   // Banana
console.log(fruits[3]);   // Orange
```

Q4. Ek empty array numbers banao aur usme 1 se 5 tak numbers push karke add karo.

Ans:

JavaScript

```
let numbers = [];
numbers.push(1, 2, 3, 4, 5);
console.log(numbers);   // [1, 2, 3, 4, 5]
```

Q5. Array ["Red", "Green", "Blue"] ke last element ko remove karo (pop) aur removed element print karo.

Ans:

JavaScript

```
let colors = ["Red", "Green", "Blue"];
let removed = colors.pop();
console.log("Removed:", removed);   // Blue
console.log("Array now:", colors);  // ["Red", "Green"]
```

Section 2: Mutation Methods (push, pop, shift, unshift, splice) (Q6-Q15)

Q6. Array [10, 20, 30] ke end mein 40 aur 50 add karo (push).

Ans:

JavaScript

```
let arr = [10, 20, 30];
arr.push(40, 50);
console.log(arr);   // [10, 20, 30, 40, 50]
```

Q7. Array [1, 2, 3, 4, 5] ke start se 2 elements remove karo (shift do baar).

Ans:

JavaScript

```
let arr = [1, 2, 3, 4, 5];
arr.shift();   // removes 1
arr.shift();   // removes 2
console.log(arr);   // [3, 4, 5]
```

Q8. Array [100, 200, 300] ke start mein 50 add karo (unshift).

Ans:

JavaScript

JavaScript Complete Notes
with real-world examples
Created by Rawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

```
let arr = [100, 200, 300];
arr.unshift(50);
console.log(arr);   // [50, 100, 200, 300]
```

Q9. Array [10, 20, 30, 40, 50] mein index 2 pe jaakar 1 element delete karo aur uski jagah 99 add karo (splice).

Ans:

JavaScript

```
let arr = [10, 20, 30, 40, 50];
arr.splice(2, 1, 99);
console.log(arr);   // [10, 20, 99, 40, 50]
```

Q10. Array [1, 2, 3, 4, 5, 6] se index 1 se 3 tak ke elements delete karo (splice).

Ans:

JavaScript

```
let arr = [1, 2, 3, 4, 5, 6];
arr.splice(1, 3);
console.log(arr);   // [1, 5, 6]
```

Q11. Array ["A", "B", "C"] ke end mein "D" aur "E" add karo, phir start se "X" add karo.

Ans:

JavaScript

```
let letters = ["A", "B", "C"];
letters.push("D", "E");
letters.unshift("X");
console.log(letters);   // ["X", "A", "B", "C", "D", "E"]
```

Q12. Array [5, 10, 15, 20] mein index 1 pe 2 elements delete karo aur unki jagah 99 aur 100 add karo.

Ans:

JavaScript

```
let arr = [5, 10, 15, 20];
arr.splice(1, 2, 99, 100);
console.log(arr);   // [5, 99, 100, 20]
```

Q13. Ek array banao aur usme 10 random numbers push karo (for loop use karke).

Ans:

JavaScript

```
let nums = [];
for (let i = 1; i <= 10; i++) {
    nums.push(Math.floor(Math.random() * 100) + 1);
}
console.log(nums);
```

Q14. Array [1, 2, 3, 4, 5] ke last element ko remove karo aur usko variable mein store karo.

Ans:

JavaScript

```
let arr = [1, 2, 3, 4, 5];
let last = arr.pop();
```

JavaScript
Learn JavaScript from basics to advanced
with real-world examples
THE ARTIFICIAL EDUCATION

```
console.log("Removed:", last);   // 5
console.log("Array:", arr);      // [1, 2, 3, 4]
```

Q15. Array ["Apple", "Banana", "Mango"] ke start mein "Orange" add karo aur end se "Mango" remove karo.

Ans:

JavaScript

```
let fruits = ["Apple", "Banana", "Mango"];
fruits.unshift("Orange");
fruits.pop();
console.log(fruits);   // ["Orange", "Apple", "Banana"]
```

Section 3: Non-Mutation Methods (slice, concat, join, toString) (Q16-Q25)

Q16. Array [10, 20, 30, 40, 50, 60] se index 1 se 4 tak ka slice nikaalo.

Ans:

JavaScript

```
let arr = [10, 20, 30, 40, 50, 60];
let sliced = arr.slice(1, 4);
console.log(sliced);   // [20, 30, 40]
```

Q17. Do arrays [1, 2, 3] aur [4, 5, 6] ko concat se join karo.

Ans:

JavaScript

```
let a = [1, 2, 3];
let b = [4, 5, 6];
let combined = a.concat(b);
console.log(combined);   // [1, 2, 3, 4, 5, 6]
```

Q18. Array ["Red", "Green", "Blue"] ko join se hyphen (-) se join karo.

Ans:

JavaScript

```
let colors = ["Red", "Green", "Blue"];
console.log(colors.join("-"));   // Red-Green-Blue
```

Q19. Array [1, 2, 3] ko toString() se string mein convert karo.

Ans:

JavaScript

```
let arr = [1, 2, 3];
console.log(arr.toString());   // "1,2,3"
```

Q20. Array ["A", "B", "C", "D"] se index 1 se end tak ka slice lo (end omit karke).

Ans:

JavaScript

```
let arr = ["A", "B", "C", "D"];
console.log(arr.slice(1));   // ["B", "C", "D"]
```

Q21. Teen arrays [10], [20, 30], [40, 50, 60] ko ek saath concat karo.

Ans:

JavaScript

JavaScript
Learn JavaScript from Basics to Advanced with real world examples
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

```
let a = [10];
let b = [20, 30];
let c = [40, 50, 60];
console.log(a.concat(b, c));   // [10, 20, 30, 40, 50, 60]
```

Q22. Array ["Hello", "World"] ko space se join karo.

Ans:

JavaScript

```
let words = ["Hello", "World"];
console.log(words.join(" "));   // "Hello World"
```

Q23. Array [1, 2, 3, 4] ka slice lo jisme sirf pehle 2 elements hon.

Ans:

JavaScript

```
let arr = [1, 2, 3, 4];
console.log(arr.slice(0, 2));   // [1, 2]
```

Q24. Array ["Apple", "Banana", "Mango", "Orange"] se sirf "Banana" aur "Mango" ka slice lo.

Ans:

JavaScript

```
let fruits = ["Apple", "Banana", "Mango", "Orange"];
console.log(fruits.slice(1, 3));   // ["Banana", "Mango"]
```

Q25. Array [5, 10, 15, 20] ko string mein convert karke comma ke saath print karo.

Ans:

JavaScript

```
let arr = [5, 10, 15, 20];
console.log(arr.toString());   // "5,10,15,20"
```

Section 4: Search & Advanced Methods (Q26-Q35)

Q26. Array [10, 20, 30, 40, 50] mein 30 ki position (indexOf) nikaalo.

Ans:

JavaScript

```
let arr = [10, 20, 30, 40, 50];
console.log(arr.indexOf(30));   // 2
```

Q27. Array ["Cat", "Dog", "Bird"] mein "Dog" hai ya nahi check karo (includes).

Ans:

JavaScript

```
let pets = ["Cat", "Dog", "Bird"];
console.log(pets.includes("Dog"));   // true
```

Q28. Array [1, 2, 3, 4, 5] ke har element ko 10 se multiply karke naya array banao (map).

Ans:

JavaScript

```
let arr = [1, 2, 3, 4, 5];
let multiplied = arr.map(x => x * 10);
console.log(multiplied);   // [10, 20, 30, 40, 50]
```

Q29. Array [10, 25, 30, 45, 60] se sirf 30 se bade numbers ka naya array banao (filter).

Ans:

JavaScript

```
let arr = [10, 25, 30, 45, 60];
let big = arr.filter(x => x > 30);
console.log(big);   // [45, 60]
```

Q30. Array [1, 2, 3, 4, 5] ka sum nikaalo (reduce).

Ans:

JavaScript

```
let arr = [1, 2, 3, 4, 5];
let sum = arr.reduce((acc, curr) => acc + curr, 0);
console.log(sum);   // 15
```

Q31. Array ["apple", "banana", "cherry"] ko alphabetically sort karo.

Ans:

JavaScript

```
let fruits = ["apple", "banana", "cherry"];
fruits.sort();
console.log(fruits);   // ["apple", "banana", "cherry"]
```

Q32. Array [5, 2, 8, 1, 9] ko ascending order mein sort karo.

Ans:

JavaScript

```
let nums = [5, 2, 8, 1, 9];
nums.sort((a, b) => a - b);
console.log(nums);   // [1, 2, 5, 8, 9]
```

Q33. Array [1, 2, 3, 4, 5] ko reverse karo.

Ans:

JavaScript

```
let arr = [1, 2, 3, 4, 5];
arr.reverse();
console.log(arr);   // [5, 4, 3, 2, 1]
```

Q34. Array [10, 20, 30] ko forEach se har element print karo.

Ans:

JavaScript

```
let arr = [10, 20, 30];
arr.forEach(item => console.log(item));
```

Q35. Array ["A", "B", "C"] ko join se pipe () separator ke saath string banao.

Ans:

JavaScript

```
let arr = ["A", "B", "C"];
console.log(arr.join("|"));   // A|B|C
```

Complete Notes
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

Section 5: Mixed / Real-world (Q36-Q40)

Q36. Ek array scores = [45, 78, 92, 65, 88] banao aur uska average nikaalo (reduce + length).

Ans:

JavaScript

```
let scores = [45, 78, 92, 65, 88];
let avg = scores.reduce((acc, curr) => acc + curr, 0) / scores.length;
console.log("Average =", avg.toFixed(2));   // 73.60
```

Q37. Array ["pen", "book", "eraser"] ke har word ko capitalize karke naya array banao (map).

Ans:

JavaScript

```
let items = ["pen", "book", "eraser"];
let capitalized = items.map(word => word[0].toUpperCase() + word.slice(1));
console.log(capitalized);   // ["Pen", "Book", "Eraser"]
```

Q38. Array [1, 2, 3, 4, 5, 6] se sirf even numbers nikaalo (filter).

Ans:

JavaScript

```
let arr = [1, 2, 3, 4, 5, 6];
let even = arr.filter(n => n % 2 === 0);
console.log(even);   // [2, 4, 6]
```

Q39. Do arrays [1, 2, 3] aur [4, 5] ko concat karke phir reverse karo.

Ans:

JavaScript

```
let a = [1, 2, 3];
let b = [4, 5];
let combined = a.concat(b).reverse();
console.log(combined);   // [5, 4, 3, 2, 1]
```

Q40. Array ["Java", "Python", "JavaScript"] ko length ke hisaab se sort karo (shortest to longest).

Ans:

JavaScript

```
let langs = ["Java", "Python", "JavaScript"];
langs.sort((a, b) => a.length - b.length);
console.log(langs);   // ["Java", "Python", "JavaScript"]
```

41. Ek array marks = [85, 97, 44, 37, 76, 60] diya hai. Class ka average marks nikaalo.

Solution:

JavaScript

```
let marks = [85, 97, 44, 37, 76, 60];
let sum = 0;

for (let val of marks) {
  sum += val;
}
```

```
let avg = sum / marks.length;
console.log("Average marks =", avg);  // 66.5
```

42. Ek array prices = [250, 645, 300, 900, 50] diya hai. Har item pe 10% offer apply karke new prices array mein save karo.

Solution:

JavaScript

```
let prices = [250, 645, 300, 900, 50];

for (let i = 0; i < prices.length; i++) {
  let offer = prices[i] * 0.10;
  prices[i] = prices[i] - offer;
}

console.log(prices);  // [225, 580.5, 270, 810, 45]
```

43. Companies array manipulations:

- First company remove karo (shift)
- "Uber" ko replace karo "Ola" se (splice)
- End mein "Amazon" add karo (push)

Solution:

JavaScript

```
let companies = ["Bloomberg", "Microsoft", "Uber", "Google", "IBM", "Netflix"];

companies.shift();            // Bloomberg removed
companies.splice(2, 1, "Ola");      // Uber → Ola
companies.push("Amazon");

console.log(companies);
```

Section 6 forEach()

(forEach har element pe callback run karta hai, kuch return nahi karta)

JavaScript

```
// 1. Simple print all elements
let numbers = [10, 20, 30, 40];
numbers.forEach(num => console.log(num));

// 2. Print with index
let fruits = ["Apple", "Banana", "Mango"];
fruits.forEach((fruit, index) => {
    console.log(`Index ${index}: ${fruit}`);
});
```

```javascript
// 3. Calculate sum using forEach
let scores = [85, 92, 78, 95];
let sum = 0;
scores.forEach(score => sum += score);
console.log("Total =", sum);   // 350

// 4. Print only even numbers
let nums = [1, 2, 3, 4, 5, 6];
nums.forEach(n => {
    if (n % 2 === 0) console.log(n);
});

// 5. Convert to uppercase
let names = ["rahul", "priya", "aman"];
names.forEach((name, i) => {
    names[i] = name.toUpperCase();   // mutation allowed
});
console.log(names);

// 6. Count how many numbers > 50
let marks = [45, 67, 89, 23, 91];
let count = 0;
marks.forEach(m => { if (m > 50) count++; });
console.log("Above 50:", count);   // 3

// 7. Print square of each number
let arr = [2, 3, 4, 5];
arr.forEach(n => console.log(n * n));

// 8. Log with custom message
let cities = ["Delhi", "Mumbai", "Bangalore"];
cities.forEach(city => console.log(`Welcome to ${city}`));

// 9. Modify object properties inside array
let students = [
    {name: "Aman", marks: 75},
    {name: "Priya", marks: 88}
];
students.forEach(student => student.marks += 5);
console.log(students);

// 10. Stop early using return (forEach cannot break, but we can use flag)
let arr2 = [1, 2, 3, 4, 5];
```

```javascript
let found = false;
arr2.forEach(num => {
    if (num === 3) found = true;
    console.log(num);
});
```

Section 7 map()

Complete Notes

(map har element ko transform karke naya array return karta hai)

JavaScript

```javascript
// 1. Double each number
let nums = [1, 2, 3, 4, 5];
let doubled = nums.map(n => n * 2);
console.log(doubled);   // [2, 4, 6, 8, 10]

// 2. Convert numbers to strings
let numbers = [10, 20, 30];
let str = numbers.map(n => n.toString());
console.log(str);   // ["10", "20", "30"]

// 3. Add 10 to each marks
let marks = [45, 67, 89];
let updated = marks.map(m => m + 10);
console.log(updated);   // [55, 77, 99]

// 4. Extract only names from array of objects
let students = [
    {name: "Rahul", age: 20},
    {name: "Priya", age: 21}
];
let names = students.map(s => s.name);
console.log(names);   // ["Rahul", "Priya"]

// 5. Convert to uppercase
let words = ["hello", "world"];
let upper = words.map(w => w.toUpperCase());
console.log(upper);   // ["HELLO", "WORLD"]

// 6. Calculate square root
let arr = [4, 9, 16, 25];
let sqrt = arr.map(Math.sqrt);
console.log(sqrt);   // [2, 3, 4, 5]
```

```javascript
// 7. Create HTML list items
let items = ["Pen", "Book", "Eraser"];
let html = items.map(item => `<li>${item}</li>`);
console.log(html.join(""));

// 8. Add index to each element
let fruits = ["Apple", "Banana"];
let withIndex = fruits.map((fruit, i) => `${i + 1}. ${fruit}`);
console.log(withIndex);

// 9. Discount 10% on prices
let prices = [100, 200, 300];
let afterDiscount = prices.map(p => p * 0.9);
console.log(afterDiscount);   // [90, 180, 270]

// 10. Boolean conversion
let values = [0, 1, "", "hello", null];
let bools = values.map(v => Boolean(v));
console.log(bools);   // [false, true, false, true, false]

// 11. Extract first letter
let names2 = ["Aman", "Priya", "Sohan"];
let firstLetters = names2.map(n => n[0]);
console.log(firstLetters);   // ["A", "P", "S"]

// 12. Create object from array
let ids = [101, 102, 103];
let users = ids.map(id => ({id: id, active: true}));
console.log(users);

// 13. Multiply by index
let arr3 = [2, 3, 4];
let result = arr3.map((num, i) => num * i);
console.log(result);   // [0, 3, 8]

// 14. Format currency
let amounts = [500, 1200, 750];
let formatted = amounts.map(a => `₹${a}`);
console.log(formatted);

// 15. Convert to object with key-value
let keys = ["name", "age", "city"];
```

THE ARTIFICIAL EDUCATION

```
let values2 = ["Rahul", 25, "Delhi"];
let obj = {};
keys.forEach((key, i) => obj[key] = values2[i]); // alternative with map
console.log(obj);
```

Section 8 filter()

(filter condition true hone wale elements ka naya array return karta hai)

JavaScript

```
// 1. Numbers greater than 50
let marks = [45, 67, 89, 23, 91, 55];
let above50 = marks.filter(m => m > 50);
console.log(above50);   // [67, 89, 91, 55]

// 2. Even numbers
let nums = [1, 2, 3, 4, 5, 6, 7, 8];
let even = nums.filter(n => n % 2 === 0);
console.log(even);   // [2, 4, 6, 8]

// 3. Students who passed (marks >= 40)
let students = [
    {name: "Aman", marks: 35},
    {name: "Priya", marks: 78},
    {name: "Rahul", marks: 92}
];
let passed = students.filter(s => s.marks >= 40);
console.log(passed);

// 4. Names starting with 'A'
let names = ["Aman", "Priya", "Ankit", "Sohan"];
let aNames = names.filter(n => n.startsWith("A"));
console.log(aNames);   // ["Aman", "Ankit"]

// 5. Prices less than 500
let prices = [200, 650, 300, 1200, 450];
let cheap = prices.filter(p => p < 500);
console.log(cheap);   // [200, 300, 450]

// 6. Non-empty strings
let arr = ["", "hello", "", "world", "js"];
let nonEmpty = arr.filter(str => str !== "");
console.log(nonEmpty);   // ["hello", "world", "js"]
```

```javascript
// 7. Adults (age >= 18)
let people = [
    {name: "Rahul", age: 17},
    {name: "Priya", age: 22},
    {name: "Aman", age: 19}
];
let adults = people.filter(p => p.age >= 18);
console.log(adults);

// 8. Positive numbers only
let mixed = [-5, 10, -3, 0, 7, -1];
let positive = mixed.filter(n => n > 0);
console.log(positive);   // [10, 7]

// 9. Words with length > 5
let words = ["apple", "banana", "cat", "elephant", "dog"];
let longWords = words.filter(w => w.length > 5);
console.log(longWords);   // ["banana", "elephant"]

// 10. Objects where property exists
let data = [
    {id: 1, active: true},
    {id: 2},
    {id: 3, active: false}
];
let activeUsers = data.filter(item => item.active === true);
console.log(activeUsers);
```

Section 9 sort()

JavaScript

```javascript
// 1. Sort numbers ascending
let nums = [40, 10, 30, 20, 50];
nums.sort((a, b) => a - b);
console.log(nums);   // [10, 20, 30, 40, 50]

// 2. Sort numbers descending
let nums2 = [40, 10, 30, 20, 50];
nums2.sort((a, b) => b - a);
console.log(nums2);   // [50, 40, 30, 20, 10]

// 3. Sort strings alphabetically
let fruits = ["Mango", "Apple", "Banana", "Orange"];
```

```
fruits.sort();
console.log(fruits);   // ["Apple", "Banana", "Mango", "Orange"]

// 4. Sort by name (array of objects)
let students = [
    {name: "Priya", marks: 85},
    {name: "Aman", marks: 92},
    {name: "Rahul", marks: 78}
];
students.sort((a, b) => a.name.localeCompare(b.name));
console.log(students);

// 5. Sort by marks descending
let students2 = [
    {name: "Aman", marks: 92},
    {name: "Priya", marks: 85},
    {name: "Rahul", marks: 78}
];
students2.sort((a, b) => b.marks - a.marks);
console.log(students2);

// 6. Sort by length of string
let words = ["cat", "elephant", "dog", "hippopotamus"];
words.sort((a, b) => a.length - b.length);
console.log(words);   // ["cat", "dog", "elephant", "hippopotamus"]

// 7. Random sort (shuffle)
let arr = [1, 2, 3, 4, 5];
arr.sort(() => Math.random() - 0.5);
console.log(arr);

// 8. Sort with multiple conditions (first by age, then by name)
let people = [
    {name: "Rahul", age: 25},
    {name: "Priya", age: 22},
    {name: "Aman", age: 25}
];
people.sort((a, b) => {
    if (a.age !== b.age) return a.age - b.age;
    return a.name.localeCompare(b.name);
});
console.log(people);
```

```
// 9. Case-insensitive sort
let names = ["apple", "Banana", "Cherry", "date"];
names.sort((a, b) => a.toLowerCase().localeCompare(b.toLowerCase()));
console.log(names);

// 10. Sort and then reverse
let scores = [45, 78, 92, 65, 88];
scores.sort((a, b) => a - b).reverse();
console.log(scores);   // [92, 88, 78, 65, 45]
```

✅ JavaScript Chapter 5: Functions –

1. Function Kya Hai? (Sabse Simple Definition)

Function ek **reusable block of code** hai jo ek specific kaam karta hai. Jaise real life mein:

- Modi ji ke paas secretary hai → meeting set karne ke liye secretary ko bulate hain.
- Driver hai → gadi chalane ke liye driver ko bulate hain.
- Cook hai → khana banane ke liye cook ko bulate hain.

Programming mein bhi aisa hi hai. Agar koi kaam baar-baar karna pad raha hai (jaise sum nikaalna, name print karna, discount calculate karna), to usko ek function mein daal dete hain. Phir jab bhi zarurat pade, function ko **call** kar dete hain.

Fayda: Code repeat nahi hota, clean rehta hai, aur baad mein change karna easy hota hai.

2. Function Kaise Likhte Hain? (Declaration)

Basic Syntax:

JavaScript

```
function functionName() {
    // yahan code likhte hain jo function karega
}
```

***Example**:*

JavaScript

```
function welcome() {
    console.log("Welcome to Artificial Education");
    console.log("We are learning JavaScript");
}

// Ab function ko call karna zaroori hai
welcome();   // Pehli baar
welcome();   // Doosri baar
```

Note: Function sirf likh dene se nahi chalta. Use **call** karna padta hai (welcome()).

3. Function Mein Input Lena (Parameters & Arguments)

Parameter: Function likhte waqt jo variable banate hain (input ke liye). **Argument**: Function call karte waqt jo value pass karte hain.

JavaScript

```
// Function definition (parameters)
function sum(a, b) {        // a aur b parameters hain
    console.log("Sum =", a + b);
}

// Function call (arguments)
sum(5, 7);      // 5 aur 7 arguments hain → Output: Sum = 12
sum(10, 20);    // Output: Sum = 30
```

Multiple parameters:

JavaScript

```
function printInfo(name, age, city) {
    console.log(`Name: ${name}, Age: ${age}, City: ${city}`);
}

printInfo("Rahul", 25, "Delhi");
```

4. Function Se Value Wapas Bhejna (Return)

return keyword use karte hain value wapas bhejne ke liye. return ke baad ka code kabhi execute nahi hota.

JavaScript

```
function multiply(x, y) {
    let result = x * y;
    return result;            // value wapas bhej di

    console.log("Yeh line kabhi nahi chalegi");   // unreachable code
}

let answer = multiply(6, 7);
console.log("Answer =", answer);   // Answer = 42
```

Agar return nahi likha to function undefined return karta hai.

5. Arrow Functions (Modern & Short Way)

Arrow functions ES6 se aaye hain. Yeh normal functions se chhote aur clean hote hain.

Syntax:

JavaScript

```
const functionName = (parameters) => {
    // code
};
```

Examples:

JavaScript

```
// 1. Basic
const greet = () => {
    console.log("Hello from Arrow Function");
};
greet();

// 2. With parameters + return
const add = (a, b) => a + b;          // single line mein return automatic
console.log(add(10, 15));             // 25

// 3. One parameter (parentheses optional)
const square = n => n * n;
console.log(square(8));               // 64

// 4. Multiple lines
const printDetails = (name, age) => {
    console.log(`Name: ${name}`);
    console.log(`Age: ${age}`);
};
printDetails("Priya", 22);
```

Normal vs Arrow:

JavaScript

```
// Normal function
function sum(a, b) {
    return a + b;
}

// Arrow function (shorter)
const sum = (a, b) => a + b;
```

Created by Pawan Soni

6. Callback Functions & Higher-Order Functions

Callback: Ek function jo dusre function ke andar argument ke roop mein pass kiya jata hai.

Learn computer, control your future

Higher-Order Function: Wo function jo dusre function ko parameter ke roop mein leta hai ya return karta hai.

Example:

JavaScript

```
let arr = [1, 2, 3, 4, 5];

arr.forEach((num) => {      // yeh callback function hai
```

THE ARTIFICIAL EDUCATION

```
    console.log(num * 2);
});
```

JavaScript mein Function Banane ke Sabhi Tarike

JavaScript mein function banane ke **4 main tarike** hain.

1. Normal / Traditional Function (Sabse Purana Tarika)

Yeh sabse common aur beginner-friendly tarika hai.

Syntax:

JavaScript

```
function functionName(parameters) {
    // code jo function karega
    // optional: return value;
}
```

Examples:

JavaScript

```
// 1. Simple function (no parameters, no return)
function greet() {
    console.log("Hello Coders!");
    console.log("Welcome to JavaScript Series");
}

greet();        // Call karna zaroori hai
greet();        // Doosri baar call
```

JavaScript

```
// 2. Function with parameters
function add(a, b) {
    console.log("Sum =", a + b);
}

add(5, 7);      // Output: Sum = 12
add(10, 20);    // Output: Sum = 30
```

JavaScript

```
// 3. Function with return
function multiply(x, y) {
    let result = x * y;
    return result;          // value wapas bhej di
}

let answer = multiply(6, 7);
console.log("Answer =", answer);   // Answer = 42
```

Note: return ke baad ka code kabhi execute nahi hota.

JavaScript Complete Notes
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

2. Function Expression (Anonymous Function)

Yahan function ko kisi variable mein store kar dete hain.

Syntax:

JavaScript

```
const variableName = function(parameters) {
    // code
};
```

Examples:

JavaScript

```
// 1. Basic function expression
const sayHello = function() {
    console.log("Hello from Function Expression!");
};

sayHello();
```

JavaScript

```
// 2. With parameters and return
const sum = function(a, b) {
    return a + b;
};

console.log(sum(8, 12));   // 20
```

Important Difference: Normal function ko declare karne se pehle bhi call kar sakte hain (hoisting). Function expression ko pehle declare karna zaroori hai, warna error aayega.

3. Arrow Function (Modern & Most Used Tarika)

ES6 se aaya hai. Sabse short aur clean tarika hai. Aajkal zyadatar developers arrow functions hi use karte hain.

Syntax:

JavaScript

```
const functionName = (parameters) => {
    // code
};
```

Different Cases with Examples:

JavaScript

```
// 1. No parameters
const greet = () => {
    console.log("Hello Arrow Function!");
};
greet();
```

JavaScript

THE ARTIFICIAL EDUCATION

```
// 2. One parameter (parentheses optional)
const square = n => n * n;
console.log(square(9));   // 81
```

JavaScript

```
// 3. Multiple parameters + return
const add = (a, b) => a + b;        // single line → return automatic
console.log(add(15, 25));           // 40
```

JavaScript

```
// 4. Multiple lines (curly braces needed)
const printDetails = (name, age) => {
    console.log(`Name: ${name}`);
    console.log(`Age: ${age}`);
};

printDetails("Priya", 22);
```

JavaScript

```
// 5. Arrow function returning object
const createUser = (name, age) => ({
    username: name,
    userAge: age,
    active: true
});

console.log(createUser("Aman", 21));
```

Arrow vs Normal Function (Quick Comparison)

Feature	Normal Function	Arrow Function
Syntax	Longer	Very short & clean
this keyword	Own this	Lexical this (parent ka this)
Used in modern code	Less	Most preferred
Hoisting	Yes	No

4. Immediately Invoked Function Expression (IIFE)

Yeh function turant call ho jata hai, declare karte hi.

Syntax:

JavaScript

```
(function() {
    // code
})();
```

Examples:

JavaScript

Complete Notes
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

```
// 1. Simple IIFE
(function() {
    console.log("This runs immediately!");
})();

// 2. IIFE with parameters
(function(name) {
    console.log("Hello " + name);
})("Rahul");        // Hello Rahul

// 3. IIFE returning value
const result = (function(a, b) {
    return a * b;
})(6, 7);

console.log(result);   // 42
```

Use case:

- Ek baar hi code run karna ho
- Private variables banana ho (scope hide karne ke liye)

25 Practical Examples

Section 1: Basic Normal Functions (Examples 1-8)

Example 1: Simple function jo welcome message print kare (no parameter, no return)

JavaScript

```
function welcome() {
    console.log("Welcome to Artificial Education!");
    console.log("We are learning JavaScript Functions");
}

welcome();   // Call karna zaroori hai
welcome();   // Doosri baar call
```

Example 2: *Function with parameters (sum of two numbers)*

JavaScript

```
function sum(a, b) {
    console.log("Sum of", a, "and", b, "is", a + b);
}

sum(10, 20);   // Sum of 10 and 20 is 30
sum(5, 7);     // Sum of 5 and 7 is 12
```

Example 3: Function with return (multiply two numbers)

JavaScript

```
function multiply(x, y) {
    return x * y;   // value wapas bhej di
```

```
}

let result = multiply(6, 8);
console.log("Multiplication result =", result);   // 48
```

Example 4: Function jo even/odd check kare (pichhle lecture ka conditional + function)

JavaScript

```
function checkEvenOdd(num) {
    if (num % 2 === 0) {
        return "Even";
    } else {
        return "Odd";
    }
}

console.log(checkEvenOdd(10));   // Even
console.log(checkEvenOdd(7));    // Odd
```

Example 5: Function jo array ke saare elements print kare (Array + Loop concept)

JavaScript

```
function printArray(arr) {
    for (let i = 0; i < arr.length; i++) {
        console.log("Element at index", i, "is", arr[i]);
    }
}

let marks = [85, 97, 64, 76];
printArray(marks);
```

Example 6: *Function jo string ko reverse kare (String + Loop)*

JavaScript

```
function reverseString(str) {
    let reversed = "";
    for (let i = str.length - 1; i >= 0; i--) {
        reversed += str[i];
    }
    return reversed;
}

console.log(reverseString("ArtificialEducation"));   // egelloCAnpA
```

Example 7: Function jo array mein se even numbers ka sum return kare

JavaScript

```
function sumOfEven(arr) {
    let sum = 0;
    for (let num of arr) {
```

```
        if (num % 2 === 0) {
            sum += num;
        }
    }
    return sum;
}

let numbers = [1, 2, 3, 4, 5, 6, 7, 8];
console.log("Sum of even numbers =", sumOfEven(numbers));   // 20
```

Example 8: Function jo user se naam le aur welcome message de (Prompt + Template Literal)

JavaScript

```
function greetUser() {
    let name = prompt("Enter your name");
    console.log(`Hello ${name}, Welcome to JavaScript Series!`);
}

greetUser();
```

Section 2: Arrow Functions (Examples 9-16)

Example 9: Basic arrow function

JavaScript

```
const greet = () => {
    console.log("Hello from Arrow Function!");
};

greet();
```

Example 10: Arrow function with parameters (addition)

JavaScript

```
const add = (a, b) => a + b;   // single line → return automatic

console.log(add(15, 25));   // 40
```

Example 11: Arrow function jo square return kare

JavaScript

```
const square = n => n * n;
console.log(square(9));   // 81
```

Example 12: Arrow function with multiple lines

JavaScript

```
const printDetails = (name, age) => {
    console.log(`Name: ${name}`);
    console.log(`Age: ${age}`);
};
```

Learn JavaScript from basics to advanced
Created by Pawan Soni
THE ARTIFICIAL EDUCATION

```
printDetails("Priya", 22);
```

Example 13: Arrow function jo array ke har element ko double kare (map concept)

JavaScript

```
const doubleArray = arr => arr.map(num => num * 2);

let nums = [1, 2, 3, 4];
console.log(doubleArray(nums));   // [2, 4, 6, 8]
```

Example 14: Arrow function jo even numbers filter kare

JavaScript

```
const getEven = arr => arr.filter(num => num % 2 === 0);

let numbers2 = [1, 2, 3, 4, 5, 6];
console.log(getEven(numbers2));   // [2, 4, 6]
```

Example 15: Arrow function jo array ka sum return kare (reduce)

JavaScript

```
const getSum = arr => arr.reduce((acc, curr) => acc + curr, 0);

let scores = [85, 92, 78, 95];
console.log("Total =", getSum(scores));   // 350
```

Example 16: Arrow function jo string ko reverse kare

JavaScript

```
const reverseStr = str => str.split('').reverse().join('');

console.log(reverseStr("JavaScript"));   // tpircSavaJ
```

Section 3: Mixed Examples (Functions + Previous Concepts) (Examples 17-25)

Example 17: Function jo username generate kare (String + Template Literal)

JavaScript

```
const generateUsername = fullName => {
    return `@${fullName.toLowerCase()}${fullName.length}`;
};

console.log(generateUsername("PawanSoni"));   // @Pawansoni14
```

Example 18: Function jo array mein se max number nikaale

JavaScript

```
const findMax = arr => {
    return arr.reduce((max, curr) => curr > max ? curr : max, arr[0]);
};

let marks = [45, 78, 92, 65, 88];
console.log("Highest marks =", findMax(marks));   // 92
```

Example 19: Function jo price pe 10% discount apply kare (Array + Map)

JavaScript

```
const applyDiscount = prices => {
    return prices.map(price => price * 0.9);
};

let prices = [100, 200, 500];
console.log(applyDiscount(prices));   // [90, 180, 450]
```

Example 20: Function jo array ko sort kare (ascending)

JavaScript

```
const sortArray = arr => {
    return [...arr].sort((a, b) => a - b);   // spread to avoid mutation
};

let scores = [78, 45, 92, 65];
console.log(sortArray(scores));   // [45, 65, 78, 92]
```

Example 21: Function jo vowel count kare (String + Loop)

JavaScript

```
const countVowels = str => {
    let count = 0;
    const vowels = "aeiouAEIOU";
    for (let char of str) {
        if (vowels.includes(char)) count++;
    }
    return count;
};

console.log(countVowels("Artificial Education"));   // 5
```

Example 22: Function jo array ke elements ko uppercase kare (Array + Map)

JavaScript

```
const toUpper = arr => arr.map(item => item.toUpperCase());

let cities = ["delhi", "mumbai", "bangalore"];
console.log(toUpper(cities));   // ["DELHI", "MUMBAI", "BANGALORE"]
```

Example 23: Function jo do arrays ko merge kare (Array + Concat)

JavaScript

```
const mergeArrays = (arr1, arr2) => arr1.concat(arr2);

let a = [1, 2, 3];
let b = [4, 5, 6];
console.log(mergeArrays(a, b));   // [1, 2, 3, 4, 5, 6]
```

Example 24: Function jo number ka factorial calculate kare (Loop + Function)

JavaScript

```
const factorial = n => {
    let fact = 1;
    for (let i = 1; i <= n; i++) {
        fact *= i;
    }
    return fact;
};

console.log(factorial(5));   // 120
```

Example 25: Function jo object ko print kare (Object + for-in loop)

JavaScript

```
const printObject = obj => {
    for (let key in obj) {
        console.log(`${key} : ${obj[key]}`);
    }
};

let student = { name: "Aman", age: 21, course: "BCA" };
printObject(student);
```

✅ DOM (Document Object Model)

1. Introduction

DOM ek programming interface hai jo browser mein HTML ya XML document ko represent karta hai. Iska matlab hai: jab browser koi webpage load karta hai, toh wo us page ke sabhi elements (tags, text, attributes) ko ek tree-like structure mein convert kar deta hai. Is tree ko hi DOM kehte hain.
Yeh tree hi DOM hai – isme har tag (element), text aur attribute ek node hota hai. **Simple words mein samjho:**

- HTML file = sirf text aur tags
- DOM = us text ko browser ne live objects mein badal diya
- Ab JavaScript is DOM tree ko access, change, delete ya add kar sakta hai

2. Kyun important hai DOM?

- Iske bina JavaScript webpage ko dynamically change nahi kar sakta
- Click karne par button color badalna, new paragraph add karna, form submit karna – sab DOM ke through hota hai

3. Window Object vs Document Object

Window Object Browser ka top-most/global object hai. Pura window/tab represent karta hai (BOM ka part). Yeh JavaScript ka root hai. Document Object Window ka child/property hai → window.document Sirf loaded HTML document + DOM tree ko represent karta hai.

Simple Comparison:

Feature	Window Object	Document Object
Represents	Browser window/tab	HTML document + DOM tree
Hierarchy	Sabse top (parent)	Window ka child (window.document)
Kaam	Browser level (alert, size, open)	Page content manipulate (elements add/change)
Examples	window.alert(), window.innerHeight	document.getElementById(), document.body

Hierarchy (simple tree):

text

```
Window (root)
   └── Document
        └── <html>
             ├── <head>
             └── <body>
```

4. DOM Tree & Nodes

DOM Tree browser mein HTML document ki hierarchical structure (ped jaisa) hoti hai. Root node se shuru hoti hai aur har cheez is tree mein branches banati hai.

Node DOM tree ka basic building block hai. Tree mein har element, text, attribute ya comment ek Node hota hai.

Main Types of Nodes:

Node Type	Kya Hai	Example
Document Node	Pura DOM ka root	document
Element Node	HTML tags	, <p>, <h1>
Text Node	Text content (tags ke andar)	"Hello World"
Attribute Node	Attributes (id, class etc.)	id="main"
Comment Node	HTML comments	<!-- comment -->

Created by **Pawan Soni**

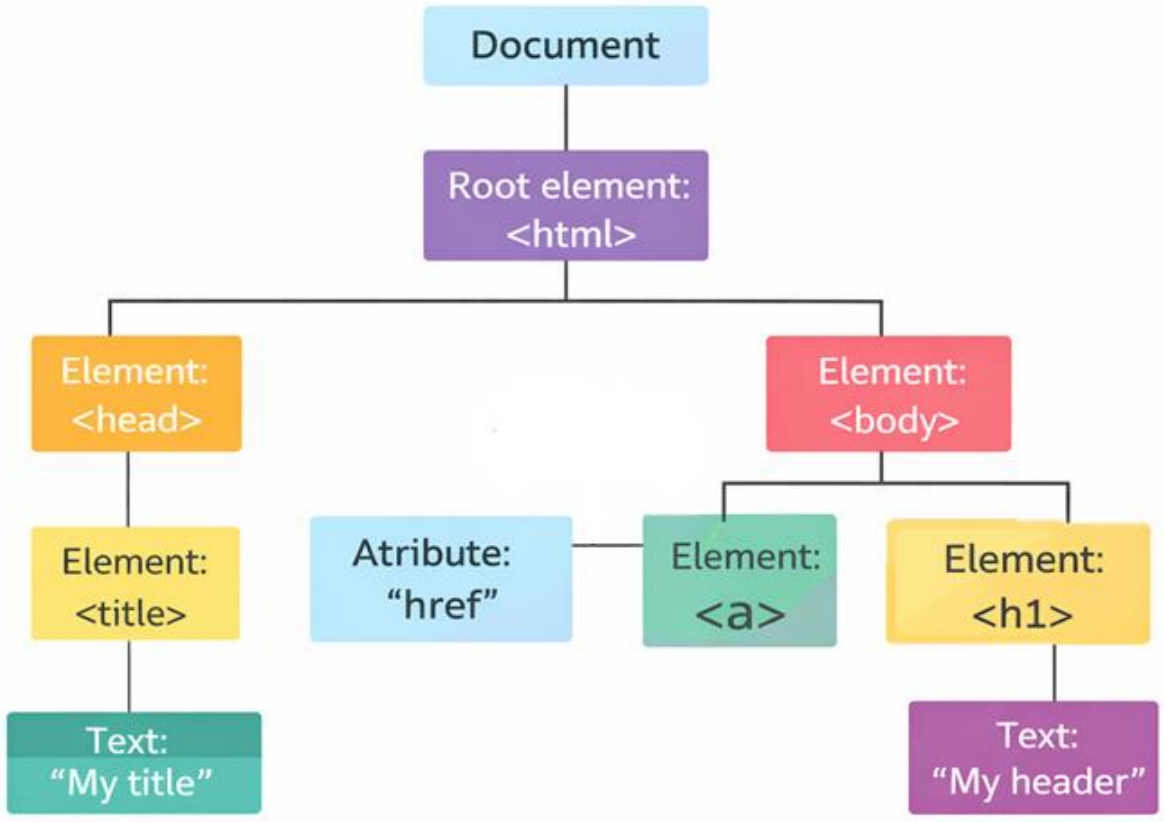
learn computer, control your future

THE ARTIFICIAL EDUCATION

Simple DOM Tree Example:

text

```
Document (Root Node)
    └── html (Element Node)
        ├── head (Element Node)
        │    └── title (Element Node)
        │         └── "DOM Tutorial" (Text Node)
        └── body (Element Node)
             ├── h1 (Element Node)
             │    └── "Welcome" (Text Node)
             └── p (Element Node)
                  └── "This is paragraph" (Text Node)
```



learn computer, control your future

THE ARTIFICIAL EDUCATION

JavaScript in nodes ko access, change ya delete kar sakta hai.

5. DOM Tree mein Parent, Child & Sibling Relationship

DOM tree mein har node ke saath Parent, Child aur Sibling relationship hoti hai. Yeh relationships JavaScript se elements ko easily navigate karne mein help karti hain.

1. Parent Node

- Node jo kisi node ke upar (direct container) hota hai.
- Har node ka ek hi parent hota hai (except root).

2. Child Node

- Node jo kisi parent ke andar (directly neeche) hota hai.
- Ek parent ke multiple children ho sakte hain.

3. Sibling Nodes

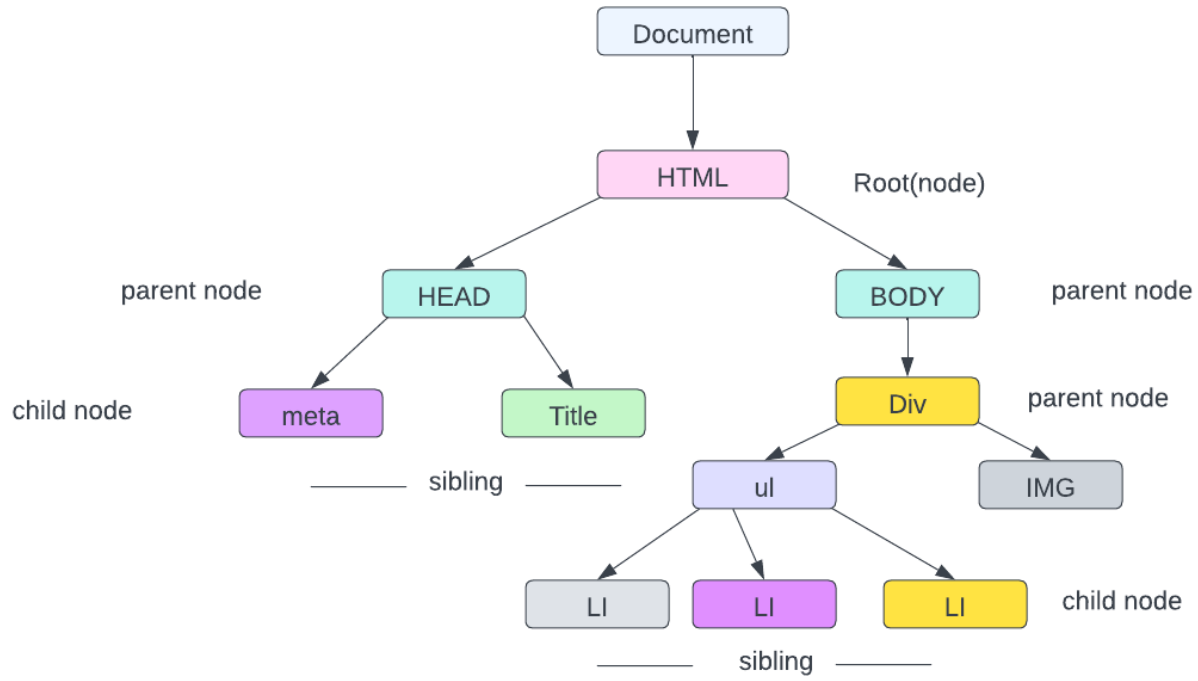
- Nodes jo same parent ke children hote hain (ek hi level par).
- Ek dusre ke bhai-behen jaise.

JavaScript
Complete Notes
learn JavaScript from basics to advanced
with real-world examples
Created by **Pawan Soni**
learn computer, control your future
THE ARTIFICIAL EDUCATION

Simple Example:

text

```
Document (Root)
    └── html
        └── body                ← Parent (sabka parent)
            ├── h1              ← Child of body + Sibling
            ├── p               ← Child of body + Sibling
            └── div             ← Child of body + Sibling
                └── span        ← Child of div
```



Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

JavaScript mein kaise use karte hain:

1. parentNode / parentElement
2. children / childNodes
3. nextElementSibling / previousElementSibling

6. Selecting / Accessing Elements in DOM

JavaScript se DOM ke elements ko select (pakadna) karne ke 5 main tareeke hain. Yeh sabse important aur commonly used methods hain.

Method	Select karta hai	Return karta hai	Example
getElementById()	ID se	Single Element	document.getElementById("title")
getElementsByClassName()	Class name se	HTMLCollection	document.getElementsByClassName("box")
getElementsByTagName()	Tag name se	HTMLCollection	document.getElementsByTagName("p")
querySelector()	CSS selector (pehle wala)	Single Element	document.querySelector(".btn")
querySelectorAll()	CSS selector (sab)	NodeList	document.querySelectorAll("div.card")

Sabse Best (Modern Way): querySelector() aur querySelectorAll() → sabse powerful aur aajkal sabse zyada use hote hain.

Simple Examples:

JavaScript

```
// ID se
let heading = document.getElementById("main");
// Class se
let boxes = document.getElementsByClassName("card");
// Modern way
let firstBtn = document.querySelector(".btn");
let allParas = document.querySelectorAll("p");
```

7. DOM Properties

DOM elements ke important properties hain jo JavaScript se read (padhna) aur modify (badalna) kar sakte ho. Yeh content, style aur attributes ko control karte hain.

Sabse Commonly Used DOM Properties:

Property	Kya Karta Hai	Example
innerHTML	Element ke andar HTML + text set/read	div.innerHTML = " New Content "

THE ARTIFICIAL EDUCATION

outerHTML	Element khud + andar ka HTML set/read	div.outerHTML = " New "
textContent	Sirf text content (safe aur fast)	p.textContent = "Hello World"
innerText	Visible text (CSS style consider karta hai)	heading.innerText
value	Form elements ka value	input.value = "Pawan"
checked	Checkbox / Radio button ka state	checkbox.checked = true
disabled	Input ko disable/enable	input.disabled = true
readonly	Input ko read-only banata hai	input.readOnly = true
placeholder	Input ka placeholder text	input.placeholder = "Enter name"
name	Form field ka name attribute	input.name = "email"
type	Input ka type	input.type = "password"
selectedIndex	Select dropdown mein selected option ka index	select.selectedIndex = 0
style	Inline CSS change	box.style.color = "red"
classList	Classes add/remove/toggle	elem.classList.add("active")
className	Puri class string set/read	elem.className = "red big"
id	Element ka ID set/read	elem.id = "main"
src	Image, script, iframe ka source	img.src = "new.jpg"
href	Anchor link ka URL	link.href = "https://example.com"
alt	Image ka alternative text	img.alt = "Profile pic"
title	Tooltip text	elem.title = "Hover karo"
dataset	data-* attributes handle	elem.dataset.userId = "123"
hidden	Element ko hide/show	elem.hidden = true
contentEditable	Element ko editable bana deta hai	div.contentEditable = true
tagName	Element ka tag name (uppercase)	console.log(div.tagName)
parentElement	Parent element milta hai	child.parentElement
children	Direct child elements	parent.children
firstElementChild	Pehla child element	parent.firstElementChild
lastElementChild	Aakhri child element	parent.lastElementChild
nextElementSibling	Agla sibling element	elem.nextElementSibling
previousElementSibling	Pehla sibling element	elem.previousElementSibling

Sabse zyada use hone wale: innerHTML / textContent → content ke liye | value → forms ke liye | style aur classList → design ke liye

8. Attributes Handling in DOM

HTML elements ke attributes (id, class, src, href, alt, disabled etc.) ko JavaScript se get, set, check aur remove karne ke methods hain.

Main Methods:

THE ARTIFICIAL EDUCATION

Method	Kya Karta Hai	Example
getAttribute()	Attribute ki value padhta hai	elem.getAttribute("href")
setAttribute()	Attribute set ya update karta hai	elem.setAttribute("class", "active")
hasAttribute()	Check karta hai attribute hai ya nahi	elem.hasAttribute("id")
removeAttribute()	Attribute delete karta hai	elem.removeAttribute("disabled")

Direct Property Way (Fast & Common): elem.id, elem.src, elem.href, elem.alt, elem.className etc.

Code Examples:

JavaScript

```
let link = document.querySelector("a");
// Get
let url = link.getAttribute("href");
// Set
link.setAttribute("target", "_blank");
// Check
if (link.hasAttribute("title")) { ... }
// Remove
link.removeAttribute("title");
// Direct
img.src = "new-image.jpg";
```

9. Styling with JavaScript

JavaScript se DOM elements ka CSS styling dynamically change kar sakte ho. Do sabse important tareeke hain:

- .style → inline styles (direct change)
- .classList → classes add/remove (best & recommended)

1. Inline Styling with .style (CamelCase use karo)

JavaScript

```
let box = document.querySelector(".box");
box.style.backgroundColor = "red";     // background-color
box.style.color = "white";
box.style.fontSize = "24px";
box.style.padding = "20px";
box.style.borderRadius = "10px";
box.style.display = "none";            // hide karne ke liye
```

2. ClassList (Best Practice) Pehle CSS mein class banao, JS se sirf toggle karo.

JavaScript

```
let box = document.querySelector(".box");
box.classList.add("active");           // class add
box.classList.remove("active");        // class remove
```

JavaScript Complete Notes
Learn JavaScript from basics to advanced
THE ARTIFICIAL EDUCATION

```
box.classList.toggle("active");        // on/off
console.log(box.classList.contains("active"));  // true/false
```

10. Creating, Inserting, Removing & Replacing Elements in DOM

JavaScript se DOM mein naya element create kar sakte ho, insert kar sakte ho, remove kar sakte ho aur replace bhi kar sakte ho.

Sabse Important Methods (Complete List)

Operation	Method	Kya Karta Hai	Example
Create	createElement()	Naya HTML element banata hai	let div = document.createElement("div")
Create Text	createTextNode()	Sirf text node banata hai	let text = document.createTextNode("Hello")
Insert (End)	append()	Last mein add (Modern & Best)	parent.append(newDiv)
Insert (Start)	prepend()	Starting mein add	parent.prepend(newDiv)
Insert (Middle)	insertBefore()	Kisi element ke pehle insert	parent.insertBefore(newDiv, existing)
Remove	remove()	Element ko directly delete (Recommended)	element.remove()
Remove (Old)	removeChild()	Parent se child delete	parent.removeChild(child)
Replace	replaceWith()	Element ko naye se replace (Modern)	oldElem.replaceWith(newElem)
Replace (Old)	replaceChild()	Traditional replace	parent.replaceChild(newElem, oldElem)

Practical Example:

JavaScript

```
// 1. Create
let newPara = document.createElement("p");
newPara.textContent = "Yeh naya paragraph hai!";
// 2. Insert
document.body.append(newPara);          // end mein
// document.body.prepend(newPara);      // start mein
// 3. Remove
newPara.remove();
// 4. Replace (agar koi purana element hai)
let oldPara = document.querySelector("p");
oldPara.replaceWith(newPara);
```

11. DOM Traversal / Navigation

DOM tree mein ek element se dusre element tak navigate karne (ghumna) ke liye traversal properties use hote hain. Yeh parent, child aur sibling ke beech movement allow karte hain.

THE ARTIFICIAL EDUCATION

Important Traversal Properties:

Property / Method	Kya Karta Hai	Example
parentElement	Parent element milta hai	child.parentElement
children	Sabhi direct child elements	parent.children
firstElementChild	Pehla child element	parent.firstElementChild
lastElementChild	Aakhri child element	parent.lastElementChild
nextElementSibling	Agla sibling element (same level)	elem.nextElementSibling
previousElementSibling	Pehla sibling element	elem.previousElementSibling

Practical Example:

JavaScript

```
let box = document.querySelector(".box");
// Parent tak jaao
let parentDiv = box.parentElement;
// Children dekho
let allKids = parentDiv.children;
// Agla sibling
let nextBox = box.nextElementSibling;
// Pehla sibling
let prevBox = box.previousElementSibling;
```

12. Event Handling Basics

Event ka simple matlab: Jab user page par kuch karta hai (click karta hai, keyboard se type karta hai, mouse ghumata hai, form submit karta hai), usko event kehte hain. Event Handling = JavaScript ko batana ki jab yeh event ho, toh yeh kaam karo.

Pehle purane tareeke the (inline onclick), lekin aaj sabse best tareeka hai addEventListener().

Kyun addEventListener() best hai?

1. Ek element par multiple events laga sakte ho
2. Code clean aur maintainable rehta hai
3. Event remove bhi kar sakte ho jab chaho

Sabse Common Events (Daily Use)

Event	Kab Trigger Hota Hai	Example Use Case
click	Mouse se click	Button press karne par
dblclick	Double click	Image zoom
mouseover	Mouse element par aaye	Hover effect
mouseout	Mouse element se bahar jaaye	Hover effect khatam
keydown	Keyboard key dabao	Game controls
keyup	Keyboard key chhodo	Search bar typing
submit	Form submit kiya	Login form

change	Input ka value badla	Dropdown change
focus / blur	Input par focus/blur	Input highlight

Practical Example (Pura Code)

HTML:

HTML

```
<button id="myBtn">Click Me</button>
```

JavaScript:

JavaScript

```
// Element select karo
let btn = document.getElementById("myBtn");
// Event listener laga do
btn.addEventListener("click", function() {
    alert("Button clicked! 🎉");
    // yahan aur bhi code likh sakte ho
});
```

Multiple events bhi laga sakte ho:

JavaScript

```
btn.addEventListener("mouseover", () => btn.style.backgroundColor = "yellow");
btn.addEventListener("mouseout", () => btn.style.backgroundColor = "blue");
```

13. Event Object (e / evt)

Jab koi event trigger hota hai (jaise click, keydown, submit), browser automatically ek special object bhejta hai handler function ke andar. Ise hum e, evt ya event naam se receive karte hain.

Yeh object event ki poori information rakhta hai – kis element par hua, kaunsa button dabaya, mouse kahan tha, etc.

Important Properties & Methods:

Property / Method	Kya Karta Hai	Example
e.target	Wo element jisme event actually hua	e.target.style.color = "red"
e.currentTarget	Wo element jisme listener laga tha	console.log(e.currentTarget)
e.type	Event ka naam	if(e.type === "click")
e.preventDefault()	Default behaviour rokta hai	e.preventDefault()
e.stopPropagation()	Event bubbling ko rokta hai	e.stopPropagation()
e.key / e.code	Keyboard key ka naam	if(e.key === "Enter")
e.clientX / e.clientY	Mouse pointer ke coordinates	console.log(e.clientX)

14. DOM Events – Complete Important List

Yeh table mein sabse important DOM events hain jo daily coding mein use hote hain. Har event ke saath "Kab Trigger Hota Hai" aur 1-2 practical examples diye hain.

1. Mouse Events

Event	Kab Trigger Hota Hai	Practical Examples (1-2)
click	Mouse se single click	Button click, Modal open, Like button

dblclick	Double click	Image zoom, Text edit mode
mousedown	Mouse button dabate hi	Drag start, Button press effect
mouseup	Mouse button chhodte hi	Drag end, Click animation complete
mouseover	Mouse element par aata hai	Hover effect (color change)
mouseout	Mouse element se bahar jaata hai	Hover effect remove
mouseenter	Mouse element ke andar enter karta hai	Dropdown menu open (better for nested elements)
mouseleave	Mouse element se completely bahar	Dropdown menu close, Tooltip hide
contextmenu	Right click	Custom right-click menu

2. Keyboard Events

Event	Kab Trigger Hota Hai	Practical Examples (1-2)
keydown	Key dabate hi (hold karne par bhi)	Game controls, Shortcut keys (Ctrl + S)
keyup	Key chhodte hi	Search as you type, Live password validation

3. Form Events

Event	Kab Trigger Hota Hai	Practical Examples (1-2)
input	Har keystroke par (real-time)	Live search bar, Password strength meter
change	Value change hone ke baad	Dropdown change, Checkbox toggle, Color picker
submit	Form submit karte waqt	Login / Registration form validation
focus	Input par cursor aata hai	Input highlight, Label animation
blur	Input se focus hat jata hai	Input validation on leaving field

4. Window & Document Events

Event	Kab Trigger Hota Hai	Practical Examples (1-2)
DOMContentLoaded	HTML DOM ready ho jaaye (fast)	JS code start karna, Elements select karna
load	Page + images, CSS sab load ho jaaye	Heavy scripts run after full page load
resize	Browser window resize kare	Responsive layout adjust, Chart resize
scroll	User scroll kare	Infinite scroll, Sticky header, Back-to-top button

15. Advanced Event Handling: Multiple Listeners & removeEventListener()

Multiple Listeners ka matlab: Ek hi element par ek se zyada event listeners laga sakte ho (same event ke liye bhi ya alag-alag events ke liye). removeEventListener() ka matlab: Jab listener ki zarurat khatam ho jaaye toh usko hata sakte ho (memory clean rakhne ke liye important hai).

Important Methods

Method	Kya Karta Hai	Example
addEventListener()	Naya listener add karta hai	btn.addEventListener("click", handler)
removeEventListener()	Listener ko remove karta hai	btn.removeEventListener("click", handler)

Note: removeEventListener() tabhi kaam karega jab same function reference use kiya ho (anonymous function nahi chalega).

Practical Examples:

JavaScript

```
let btn = document.querySelector("button");
// 1. Multiple Listeners (same event)
btn.addEventListener("click", function() {
    console.log("Listener 1: Button clicked");
});
btn.addEventListener("click", function() {
    console.log("Listener 2: Another action");
});
// 2. Named function (remove karne ke liye zaruri)
function handleClick(e) {
    console.log("Button clicked!", e.target);
}
btn.addEventListener("click", handleClick);   // add
// Kabhi bhi remove kar sakte ho
btn.removeEventListener("click", handleClick); // remove
```

Advanced Options (Extra Power):

JavaScript

```
btn.addEventListener("click", handler, {
    once: true,       // sirf ek baar chalega
    passive: true,    // performance ke liye (scroll events mein best)
    capture: false    // bubbling phase
});
```

16. Event Delegation

Pehle Event Bubbling aur Capturing ko bilkul clear karte hain, phir Event Delegation samjhaata hoon.

1. Event Propagation (Bubbling + Capturing) Jab koi event trigger hota hai (jaise button par click), toh event 3 phases mein chalta hai:

THE ARTIFICIAL EDUCATION

Phase	Direction	Kya Hota Hai
Capturing	Top se neeche (Document → Target)	Event root se target tak jaata hai
Target	Actual element par	Event target element par pahunchta hai
Bubbling	Neeche se upar (Target → Document)	Event target se parent elements tak bubble karta hai (sabse important)

Bubbling (Default behaviour): Event target element se shuru hokar uske parent, grand-parent, body, html, document tak jaata hai.

Rokne ke liye:

- e.stopPropagation() → bubbling ya capturing rok deta hai
- e.stopImmediatePropagation() → aur bhi listeners ko rok deta hai

2. Event Delegation – Kya Hai aur Kyun Best Practice? Event Delegation ka simple idea: Har child element par alag-alag listener mat lagao. Sirf ek parent par listener laga do aur e.target se check karo ki kaunsa child click hua.

Performance Fayda:

- 1000 buttons hain toh 1000 listeners nahi, sirf 1 listener lagega → memory aur speed dono mein bahut better
- Dynamic elements (jaise list mein naye items add hote hain) ke liye perfect

Practical Example (List mein buttons):

HTML

HTML

```html
<ul id="parent">
  <li><button>Item 1</button></li>
  <li><button>Item 2</button></li>
  <!-- 1000 items bhi ho sakte hain -->
</ul>
```

JavaScript

JavaScript

```javascript
// Sirf parent par ek listener
document.getElementById("parent").addEventListener("click", function(e) {

    // Check karo actual click kahan hua
    if (e.target.tagName === "BUTTON") {
        console.log("Button clicked:", e.target.textContent);
        // yahan koi action
    }
});
```

DOM Related 20 Practical Questions

1. Ek element ko ID se select karke uska text change karo.

JavaScript

JavaScript Complete Notes
Learn JavaScript from basics to advanced with real-world examples
THE ARTIFICIAL EDUCATION

```
// Question: Button click karne par heading ka text "Hello DOM!" ho jaaye
document.getElementById("myBtn").addEventListener("click", () => {
  document.getElementById("heading").textContent = "Hello DOM!";
});
```

2. Class name se saare boxes ko select karke unka background red kar do.

JavaScript

```
const boxes = document.getElementsByClassName("box");
for(let box of boxes) {
  box.style.backgroundColor = "red";
}
```

3. QuerySelectorAll se saare paragraphs select karke unme "New Text" add karo.

JavaScript

```
document.querySelectorAll("p").forEach(p => {
  p.textContent += " - New Text";
});
```

4. Ek input field ka value console mein print karo jab user type kare.

JavaScript

```
document.getElementById("name").addEventListener("input", (e) => {
  console.log("Current value:", e.target.value);
});
```

5. Ek image ka source change karo button click par.

JavaScript

```
document.getElementById("changeImg").addEventListener("click", () => {
  document.getElementById("myImg").src = "https://picsum.photos/300";
});
```

6. Naya paragraph create karke body ke end mein add karo.

JavaScript

```
const newP = document.createElement("p");
newP.textContent = "Yeh naya paragraph hai!";
newP.style.color = "blue";
document.body.appendChild(newP);
```

7. Ek list item create karke ul ke starting mein prepend karo.

JavaScript

```
const li = document.createElement("li");
li.textContent = "First Item";
document.getElementById("myList").prepend(li);
```

8. Kisi element ko remove karo button click par.

JavaScript

```
document.getElementById("removeBtn").addEventListener("click", () => {
  document.getElementById("oldDiv").remove();
});
```

9. Ek div ko replace kar do naye div se.

JavaScript

```
const old = document.getElementById("old");
const newDiv = document.createElement("div");
newDiv.textContent = "Replaced!";
old.replaceWith(newDiv);
```

10. Parent element tak jaao aur uska background change karo.

JavaScript

```
const child = document.querySelector(".child");
const parent = child.parentElement;
parent.style.backgroundColor = "yellow";
```

11. Button par click event lagaao aur alert dikhao.

JavaScript

```
document.getElementById("btn").addEventListener("click", () => {
  alert("Button Clicked! 🎉");
});
```

12. Mouse hover par box ka color change karo (mouseover + mouseout).

JavaScript

```
const box = document.getElementById("box");
box.addEventListener("mouseover", () => box.style.backgroundColor = "orange");
box.addEventListener("mouseout", () => box.style.backgroundColor = "lightblue");
```

13. Form submit hone par default behaviour rok do aur message dikhao.

JavaScript

```
document.getElementById("myForm").addEventListener("submit", (e) => {
  e.preventDefault();
  alert("Form submitted successfully!");
});
```

14. Keyboard se Enter dabane par search trigger ho.

JavaScript

```
document.getElementById("search").addEventListener("keydown", (e) => {
  if(e.key === "Enter") {
    console.log("Search triggered for:", e.target.value);
  }
});
```

15. Event Delegation use karke list ke har item par click handle karo.

HTML

```
<ul id="list">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
```

JavaScript

```
document.getElementById("list").addEventListener("click", (e) => {
  if(e.target.tagName === "LI") {
```

```
    alert("You clicked: " + e.target.textContent);
  }
});
```

16. Ek element ke next sibling ko select karke uska text bold kar do.

JavaScript

```
const current = document.getElementById("current");
current.nextElementSibling.style.fontWeight = "bold";
```

17. Sabse pehle child element ko hide kar do.

JavaScript

```
const parent = document.getElementById("parent");
parent.firstElementChild.style.display = "none";
```

18. ClassList use karke active class toggle karo.

JavaScript

```
document.getElementById("menu").addEventListener("click", () => {
  document.getElementById("menu").classList.toggle("active");
});
```

19. Data attribute se value read karke console mein print karo.

HTML

```
<div id="user" data-user-id="786" data-name="Pawan"></div>
```

JavaScript

```
const user = document.getElementById("user");
console.log(user.dataset.userId);     // 786
console.log(user.dataset.name);       // Pawan
```

20. Ek button click karne par pura page background random color kar do.

JavaScript

```
document.getElementById("randomBtn").addEventListener("click", () => {
  const r = Math.floor(Math.random() * 256);
  const g = Math.floor(Math.random() * 256);
  const b = Math.floor(Math.random() * 256);
  document.body.style.backgroundColor = `rgb(${r},${g},${b})`;
});
```

Created by Pawan Soni

✅ Date And Time

1. Current Date + Time Kaise Lete Hain? (Sabse Common)

JavaScript

```
// Current date aur time
const now = new Date();
console.log(now);                    // Example: 2026-04-05T05:12:34.123Z
```

2. Important Methods (Sabse zyada use hone wale)

Method	Kya Karta Hai	Example Output

THE ARTIFICIAL EDUCATION

getFullYear()	Saal (4 digit)	2026
getMonth()	Mahina (0 = Jan, 11 = Dec)	3 (April ke liye)
getDate()	Din (1-31)	5
getHours()	Ghanta (0-23)	10
getMinutes()	Minute (0-59)	42
getSeconds()	Second (0-59)	15
toISOString()	API ke liye best format (UTC)	"2026-04-05T05:12:34.123Z"
toLocaleString()	Local time + date (readable)	"5/4/2026, 10:42:34 AM"

Practical Examples (Copy-Paste Kar Ke Try Kar)

JavaScript

```javascript
// 1. Current date-time ko alag-alag parts mein nikaalna
const date = new Date();

console.log("Saal:", date.getFullYear());
console.log("Mahina:", date.getMonth() + 1);     // +1 kyuki 0-based hota hai
console.log("Din:", date.getDate());
console.log("Time:",
`${date.getHours()}:${date.getMinutes()}:${date.getSeconds()}`);

// 2. API ke liye date bhejna (sabse important!)
const payload = {
  name: "Rahul",
  created_at: new Date().toISOString(),     // ← Ye sabse best hai
  // ya agar sirf date chahiye
  dob: "1995-12-25"
};

console.log(payload);

// 3. Custom format (DD/MM/YYYY)
function formatDate(date) {
  const d = new Date(date);
  const day = String(d.getDate()).padStart(2, '0');
  const month = String(d.getMonth() + 1).padStart(2, '0');
  const year = d.getFullYear();
  return `${day}/${month}/${year}`;
}

console.log(formatDate(new Date()));   // Output: 05/04/2026
```

1. Current Date-Time ko DD/MM/YYYY HH:MM:SS format mein print karo.

Solution:

JavaScript Complete Notes
Learn JavaScript from basics to advanced
THE ARTIFICIAL EDUCATION

JavaScript

```
const now = new Date();

const day = String(now.getDate()).padStart(2, '0');
const month = String(now.getMonth() + 1).padStart(2, '0');
const year = now.getFullYear();
const hours = String(now.getHours()).padStart(2, '0');
const minutes = String(now.getMinutes()).padStart(2, '0');
const seconds = String(now.getSeconds()).padStart(2, '0');

console.log(`${day}/${month}/${year} ${hours}:${minutes}:${seconds}`);
```

Example Output: 05/04/2026 10:52:34 **Explanation:** padStart(2, '0') se single digit ko 01, 02 banate hain.

2. Aaj ka date ISO format (YYYY-MM-DDTHH:mm:ss.sssZ) mein nikaalo (API ke liye best).

Solution:

JavaScript

```
const now = new Date();
console.log(now.toISOString());
```

Example Output: "2026-04-05T10:52:34.123Z" **Explanation:** toISOString() sabse standard format hai jo backend/API ko directly bheja ja sakta hai.

3. Birthdate "1995-12-25" se user ki age (years only) calculate karo.

Solution:

JavaScript

```
const birthDate = new Date("1995-12-25");
const today = new Date();

let age = today.getFullYear() - birthDate.getFullYear();

if (today.getMonth() < birthDate.getMonth() ||
   (today.getMonth() === birthDate.getMonth() && today.getDate() <
birthDate.getDate())) {
    age--;
}

console.log(`Age: ${age} years`);
```

Example Output: Age: 30 years **Explanation:** Month aur date check karna zaroori hai warna age galat ho sakti hai.

4. Current time ko 12-hour format mein AM/PM ke saath print karo.

Solution:

JavaScript

```
const now = new Date();
let hours = now.getHours();
const minutes = String(now.getMinutes()).padStart(2, '0');
```

```
const seconds = String(now.getSeconds()).padStart(2, '0');
const ampm = hours >= 12 ? 'PM' : 'AM';

hours = hours % 12;
hours = hours ? hours : 12;   // 0 ko 12 banao

console.log(`${hours}:${minutes}:${seconds} ${ampm}`);
```

Example Output: 10:52:34 AM **Explanation:** % 12 aur AM/PM logic 12-hour format banata hai.

5. Current date ko 7 din aage badha do.

Solution:

JavaScript

```
const date = new Date();
date.setDate(date.getDate() + 7);

console.log("7 din baad:", date.toLocaleDateString('en-IN'));
```

Example Output: 12/04/2026 **Explanation:** setDate() original date ko modify kar deta hai.

6. Aaj se 31 December 2026 tak kitne din baaki hain?

Solution:

JavaScript

```
const today = new Date();
const target = new Date("2026-12-31");

const diffTime = target - today;
const diffDays = Math.ceil(diffTime / (1000 * 60 * 60 * 24));

console.log(`${diffDays} days remaining`);
```

Explanation: Date objects ko subtract karne se milliseconds milte hain, phir divide karke days nikaalte hain.

7. API se aaya date "2026-04-05T10:52:34.123Z" ko Indian readable format mein convert karo.

Solution:

JavaScript

```
const apiDate = new Date("2026-04-05T10:52:34.123Z");

const options = {
    day: '2-digit',
    month: 'long',
    year: 'numeric',
    hour: '2-digit',
    minute: '2-digit',
    hour12: true
};
```

Learn JavaScript from basics to advanced
THE ARTIFICIAL EDUCATION

```
console.log(apiDate.toLocaleString('en-IN', options));
```

Example Output: 05 April 2026, 04:12 PM **Explanation:** toLocaleString best way hai readable Indian format ke liye.

8. 31 December 2026 tak ka live countdown timer banao (Days : Hours : Minutes : Seconds).

Solution:

JavaScript

```
const targetDate = new Date("2026-12-31 23:59:59").getTime();

const countdown = setInterval(() => {
    const now = new Date().getTime();
    const distance = targetDate - now;

    if (distance < 0) {
        clearInterval(countdown);
        console.log("🎉 Countdown Finished!");
        return;
    }

    const days = Math.floor(distance / (1000 * 60 * 60 * 24));
    const hours = Math.floor((distance % (1000 * 60 * 60 * 24)) / (1000 * 60 *
60));
    const minutes = Math.floor((distance % (1000 * 60 * 60)) / (1000 * 60));
    const seconds = Math.floor((distance % (1000 * 60)) / 1000);

    console.log(`⏳ ${days}d ${hours}h ${minutes}m ${seconds}s`);
}, 1000);
```

Explanation: setInterval + clearInterval se live countdown chalta hai.

9. User ne date picker se date select ki. Us date ko next month ke first day pe set kar do.

Solution:

JavaScript

```
let selectedDate = new Date("2026-04-05");   // user ki selected date

selectedDate.setMonth(selectedDate.getMonth() + 1);
selectedDate.setDate(1);

console.log("Next month first day:", selectedDate.toISOString().split('T')[0]);
```

Example Output: 2026-05-01 **Explanation:** setMonth(+1) aur setDate(1) next month ka pehla din set karta hai.

10. 5 second baad ek message show karo aur usko clearTimeout se cancel karne ka option bhi rakho.

Solution:

JavaScript Complete Notes
THE ARTIFICIAL EDUCATION

JavaScript

```
let timeoutId = setTimeout(() => {
    console.log("✅ Order Placed Successfully!");
}, 5000);

// 2 second baad cancel kar do (demo ke liye)
setTimeout(() => {
    clearTimeout(timeoutId);
    console.log("❌ Timeout cancelled by user");
}, 2000);
```

Explanation: setTimeout one-time delay deta hai aur clearTimeout se rok sakte hain.

✅ Timeout

1. setTimeout() – Ek Baar Execution (Delay ke Baad)

Kya hai? Yeh function ek baar code ko **delay ke baad** chalata hai. Baar-baar nahi chalta.

Syntax:

JavaScript

```
const timeoutId = setTimeout(function, delay, arg1, arg2, ...);
```

Practical Example (Date & Time ke saath):

JavaScript

```
// 3 second baad ek message show karo
setTimeout(() => {
    const now = new Date();
    console.log("3 second baad time:", now.toLocaleTimeString());
}, 3000);   // 3000 ms = 3 seconds
```

Real use:

- Button click ke baad 2 second baad form submit karna
- Loading spinner 1.5 second ke baad hide karna
- "Welcome" toast message delay se dikhana

2. clearTimeout() – setTimeout ko Rokna

Kya hai? Agar tumhe setTimeout ko cancel karna ho (pehle hi rokna ho) toh yeh use karte ho.

Syntax:

JavaScript

```
clearTimeout(timeoutId);
```

Practical Example:

JavaScript

```
let timeoutId = setTimeout(() => {
    console.log("Yeh message nahi dikhega");
}, 5000);

// 2 second baad timeout cancel kar do
```

```
setTimeout(() => {
    clearTimeout(timeoutId);
    console.log("Timeout cancel ho gaya!");
}, 2000);
```

3. setInterval() – Baar-Baar Execution (Har Interval Pe)

Kya hai? Yeh function code ko **har fixed time** ke baad repeat karta rahta hai (jaise live clock).

JavaScript

```
const intervalId = setInterval(function, delay);
```

Practical Example (Live Clock – Date & Time):

JavaScript

```
const clockInterval = setInterval(() => {
    const now = new Date();
    console.log(`Live Time:
${now.getHours()}:${now.getMinutes()}:${now.getSeconds()}`);
}, 1000);
```

4. clearInterval() – setInterval ko Rokna

Kya hai? Jab interval ko hamesha ke liye band karna ho tab yeh use karte ho (bahut zaroori hai warna memory leak ho jata hai).

Syntax:

JavaScript

```
clearInterval(intervalId);
```

Practical Example (Countdown with Stop):

JavaScript

```
let count = 10;
const countdownInterval = setInterval(() => {
    console.log("Time left:", count);
    count--;

    if (count < 0) {
        clearInterval(countdownInterval);   // ← Important!
        console.log("🎉 Time's Up!");
    }
}, 1000);
```

Sabka Ek Saath Comparison Table (Sabse Important)

Function	Kitni Baar Chalta Hai	Rukne Ka Tarika	Best Use Case (Date/Time)
setTimeout()	Sirf **1 baar**	clearTimeout()	Delay message, one-time action
setInterval()	**Baar-baar**	clearInterval()	Live clock, countdown, real-time update
clearTimeout()	—	Timeout cancel	User ne action liya toh delay cancel

THE ARTIFICIAL EDUCATION

clearInterval()	—	Interval stop	Countdown khatam hone pe ya page unload pe

10 Practical Questions

Yeh questions **real-life projects** ke hisaab se banaye gaye hain. Har question ke saath **complete working code**, **example output** aur **explanation** diya hai.

1. Button click karne ke 3 second baad "Order Placed!" message show karo (setTimeout).

Solution:

JavaScript

```
function placeOrder() {
    setTimeout(() => {
        console.log("✅ Order Placed Successfully!");
        alert("Order Placed!");
    }, 3000);
}

// Usage: <button onclick="placeOrder()">Place Order</button>
```

Explanation: setTimeout sirf **ek baar** delay ke baad code chalata hai.

2. 5 second baad message show hone wala tha, lekin user ne 2 second mein cancel kar diya (clearTimeout).

Solution:

JavaScript

```
let timeoutId;

function startOrder() {
    timeoutId = setTimeout(() => {
        console.log("✅ Order Placed!");
    }, 5000);
    console.log("Order processing started...");
}

function cancelOrder() {
    clearTimeout(timeoutId);
    console.log("❌ Order Cancelled by user");
}
```

Explanation: clearTimeout se delay wala action rok sakte ho.

3. Har 1 second mein current time live update karo (Live Digital Clock).

Solution:

JavaScript

```
const clock = setInterval(() => {
    const now = new Date();
    console.log(`🕒 Live Time:
```

```
${now.getHours()}:${now.getMinutes()}:${now.getSeconds()}`);
}, 1000);
```

Explanation: setInterval baar-baar repeat karta hai.

4. Har 2 second mein random number (1-100) print karo, lekin 10 second baad automatically stop kar do.

Solution:

JavaScript

```
let count = 0;
const intervalId = setInterval(() => {
    const num = Math.floor(Math.random() * 100) + 1;
    console.log("Random Number:", num);

    count++;
    if (count >= 5) {           // 5 × 2 sec = 10 sec
        clearInterval(intervalId);
        console.log("✅ Stopped after 10 seconds");
    }
}, 2000);
```

Explanation: clearInterval se interval ko band kar dete hain.

5. 31 December 2026 tak ka live countdown timer banao.

Solution:

JavaScript

```
const targetDate = new Date("2026-12-31 23:59:59").getTime();

const countdown = setInterval(() => {
    const now = new Date().getTime();
    const distance = targetDate - now;

    if (distance < 0) {
        clearInterval(countdown);
        console.log("🎉 Happy New Year 2027!");
        return;
    }

    const days = Math.floor(distance / (1000 * 60 * 60 * 24));
    const hours = Math.floor((distance % (1000 * 60 * 60 * 24)) / (1000 * 60 *
60));
    const minutes = Math.floor((distance % (1000 * 60 * 60)) / (1000 * 60));
    const seconds = Math.floor((distance % (1000 * 60)) / 1000);

    console.log(`⏳ ${days}d ${hours}h ${minutes}m ${seconds}s`);
}, 1000);
```

Explanation: Countdown mein setInterval + clearInterval dono use hote hain.

6. Form auto-save har 30 second mein (background mein).

Solution:

JavaScript

```
let autoSave = setInterval(() => {
    console.log("📝 Form Auto-Saved at", new Date().toLocaleTimeString());
    // Yahan API call kar sakte ho save ke liye
}, 30000);

// Stop karne ke liye
// clearInterval(autoSave);
```

Explanation: Real apps mein data loss se bachane ke liye use hota hai.

7. User ne "Start Live Updates" button click kiya → har 3 second mein new notification check karo.

Solution:

JavaScript

```
let liveInterval;

function startLiveUpdates() {
    liveInterval = setInterval(() => {
        console.log("🔔 New Notification Checked at", new
Date().toLocaleTimeString());
    }, 3000);
}

function stopLiveUpdates() {
    clearInterval(liveInterval);
    console.log("⏹ Live Updates Stopped");
}
```

Explanation: Start/Stop button ke liye setInterval aur clearInterval ka perfect combination.

8. 8 second baad ek welcome message show karo, lekin user chahe toh 4 second mein cancel kar sake.

Solution:

JavaScript

```
let welcomeTimeout = setTimeout(() => {
    console.log("👋 Welcome to the dashboard!");
}, 8000);

function cancelWelcome() {
    clearTimeout(welcomeTimeout);
    console.log("❌ Welcome message cancelled");
}
```

Explanation: clearTimeout se one-time delay ko rok sakte ho.

9. Ek button banao jo click karne pe 5 random numbers (1 second interval pe) generate kare aur phir khud stop ho jaaye.

Solution:

JavaScript

```
function startRandomGenerator() {
    let i = 0;
    const gen = setInterval(() => {
        console.log("Random:", Math.floor(Math.random() * 100) + 1);
        i++;
        if (i >= 5) {
            clearInterval(gen);
            console.log("✅ Generator Stopped");
        }
    }, 1000);
}
```

Explanation: Limited times tak interval chalana.

10. Ek complete "Start / Stop" Live Clock with manual stop button.

Solution (HTML + JS):

HTML

```
<button onclick="startClock()">Start Clock</button>
<button onclick="stopClock()">Stop Clock</button>
<div id="liveClock"></div>

<script>
let clockInterval;

function startClock() {
    clockInterval = setInterval(() => {
        const now = new Date();
        document.getElementById("liveClock").innerHTML =
            `🕒 ${now.getHours()}:${now.getMinutes()}:${now.getSeconds()}`;
    }, 1000);
}

function stopClock() {
    clearInterval(clockInterval);
    console.log("⏹ Clock Stopped");
}
</script>
```

Explanation: Real web projects mein yahi pattern sabse zyada use hota hai.

THE ARTIFICIAL EDUCATION

JavaScript

Complete Notes

Learn **JavaScript** from basics to advanced
with real-world examples

Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

✅ Math Object in JavaScript

1. Math Object ke Important Constants (Fixed Values)

Constant	Matlab	Example Value
Math.PI	π (pi)	3.141592653589793
Math.E	Euler's number	2.718281828459045
Math.SQRT2	Square root of 2	1.414213562373095

2. Sabse Important Math Methods (Table)

Method	Kya Karta Hai	Practical Example
Math.round(x)	Nearest integer (0.5 up)	Math.round(4.7) → 5
Math.ceil(x)	Upward round (ceiling)	Math.ceil(4.1) → 5
Math.floor(x)	Downward round	Math.floor(4.9) → 4
Math.trunc(x)	Decimal part hata deta hai	Math.trunc(4.999) → 4
Math.abs(x)	Positive value	Math.abs(-10) → 10
Math.pow(base, exp)	Power (base^exp)	Math.pow(2, 3) → 8
Math.sqrt(x)	Square root	Math.sqrt(16) → 4
Math.max(a,b,...)	Sabse badi value	Math.max(5, 10, 2) → 10
Math.min(a,b,...)	Sabse chhoti value	Math.min(5, 10, 2) → 2
Math.random()	**0 se 1 ke beech random number** (sabse important)	Math.random() → 0.7854...

3. Sabse Useful Practical Examples

JavaScript

```javascript
// 1. Basic Calculations
console.log(Math.PI);                    // 3.14159...
console.log(Math.pow(5, 2));             // 25
console.log(Math.sqrt(81));              // 9
console.log(Math.abs(-25));              // 25

// 2. Rounding (bahut use hota hai price, rating mein)
console.log(Math.round(4.3));   // 4
console.log(Math.round(4.7));   // 5
console.log(Math.ceil(4.1));    // 5
console.log(Math.floor(4.9));   // 4

// 3. Random Number Generator (Sabse Powerful)
console.log(Math.random());                    // 0 to 1

// Random integer 1 se 100 tak
```

```
const randomNum = Math.floor(Math.random() * 100) + 1;
console.log(randomNum);

// Random integer between min aur max (super useful)
function randomBetween(min, max) {
    return Math.floor(Math.random() * (max - min + 1)) + min;
}

console.log(randomBetween(10, 50));   // 10 to 50 ke beech random
```

Practical Questions

1. Ek number 7.89 ko round, ceil aur floor kar ke alag-alag print karo.

Solution:

JavaScript

```
const num = 7.89;
console.log("Round:", Math.round(num));     // 8
console.log("Ceil:", Math.ceil(num));       // 8
console.log("Floor:", Math.floor(num));     // 7
```

Explanation: round nearest integer deta hai, ceil upar ki taraf, floor neeche ki taraf.

2. 1 se 100 tak random integer generate karo.

Solution:

JavaScript

```
const random = Math.floor(Math.random() * 100) + 1;
console.log("Random 1-100:", random);
```

Explanation: Math.random() 0 se 1 tak deta hai. *100 se 0-99, +1 se 1-100, Math.floor integer banata hai.

3. Function banao jo minimum aur maximum ke beech random integer de (min=10, max=50).

Solution:

JavaScript

```
function randomBetween(min, max) {
    return Math.floor(Math.random() * (max - min + 1)) + min;
}

console.log(randomBetween(10, 50));
```

Explanation: Yeh formula sabse common hai random range ke liye. Games aur simulations mein bahut use hota hai.

4. 5^3 (5 ki power 3) aur square root of 81 calculate karo.

Solution:

JavaScript

```
console.log("5^3 =", Math.pow(5, 3));     // 125
console.log("√81 =", Math.sqrt(81));      // 9
```

Explanation: pow(base, exponent) power calculate karta hai, sqrt square root.

5. Array [5, 12, 8, 25, 3] se sabse badi aur sabse chhoti value nikaalo.

Solution:

JavaScript

```
const arr = [5, 12, 8, 25, 3];
console.log("Max:", Math.max(...arr));   // 25
console.log("Min:", Math.min(...arr));   // 3
```

Explanation: Spread operator ... array ko arguments mein convert karta hai.

6. 4-digit OTP generate karo (jaise banking apps mein).

Solution:

JavaScript

```
const otp = Math.floor(1000 + Math.random() * 9000);
console.log("Your OTP:", otp);   // Example: 7842
```

Explanation: 1000 + random*9000 se 1000-9999 range milta hai. Real apps mein yahi logic use hota hai.

7. Radius = 7 wale circle ka area calculate karo (use Math.PI).

Solution:

JavaScript

```
const radius = 7;
const area = Math.PI * Math.pow(radius, 2);
console.log("Circle Area:", area.toFixed(2));   // ~153.94
```

Explanation: Formula = πr^2. toFixed(2) decimal tak limit karta hai.

8. Negative number -45.67 ko positive banao aur phir round karo.

Solution:

JavaScript

```
const num = -45.67;
console.log("Positive:", Math.abs(num));           // 45.67
console.log("Rounded:", Math.round(Math.abs(num))); // 46
```

Explanation: abs() negative ko positive banata hai.

9. Ek 6-faced dice roller banao (button click pe 1-6 random number).

Solution (HTML + JS):

HTML

```
<button onclick="rollDice()">Roll Dice 🎲</button>
<div id="result"></div>

<script>
function rollDice() {
    const dice = Math.floor(Math.random() * 6) + 1;
    document.getElementById("result").innerHTML = `You rolled: <b>${dice}</b>`;
}
</script>
```

Explanation: Games aur gambling apps mein yahi logic use hota hai.

10. Har 2 second mein ek random number (1-100) generate karke console mein print karo (use setInterval). 10 second baad stop kar do.

Solution:

Complete Notes
THE ARTIFICIAL EDUCATION

JavaScript

```javascript
let count = 0;
const interval = setInterval(() => {
    const num = Math.floor(Math.random() * 100) + 1;
    console.log("Random number:", num);

    count++;
    if (count >= 5) {                // 5 times = 10 seconds
        clearInterval(interval);
        console.log("✅ Stopped!");
    }
}, 2000);
```


Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

✅ OOP (Object Oriented Programming)

OOP Kya Hai?

OOP ek programming style hai jisme hum real-world cheezon ko code mein model karte hain. Hum cheezon ko objects mein banaate hain. **Real-life analogy:** Duniya mein har cheez ek object hai → Car, Mobile, Student, Dog. Har object ke paas properties (features) aur behaviours (kaam) hote hain.

2. Object Kya Hai? (Sabse Basic)

Definition: Object ek entity (cheez) hai jiske andar data (properties) aur functions (methods) dono hote hain.

- Properties = uski khaasiyat / state (naam, marks, color)
- Methods = uska behaviour / kaam (start(), stop())

Real-life example: Car ek object hai

- Properties → brand, model, color, year
- Methods → start(), stop(), accelerate()

JavaScript mein Object Literal:

JavaScript

```
const car = {
    brand: "Tata",
    model: "Nexon",
    color: "Black",
    year: 2024,
    start: function() {
        console.log(    .brand + " " +     .model + " started! 🚗");
    }
};
console.log(car.brand); // Dot notation
console.log(car["model"]); // Bracket notation
car.start();
```

Summary: Object = Properties + Methods ka collection.

3. Class Kya Hai?

Definition: Class ek blueprint (naksha / template) hota hai jisse hum multiple similar objects bana sakte hain. **Real-life analogy:** Car Factory mein ek blueprint hota hai → usi se 1000 cars ban jaati hain. **Class = Blueprint Object = Blueprint se bani actual car**

JavaScript mein Class (ES6):

JavaScript

```
class Car {
    constructor(brand, model) {
            .brand = brand;
            .model = model;
    }
    start() {
```

```
        console.log(`${    .brand} ${    .model} started! 🚗`);
    }
}
```

Objects banana:

JavaScript

```
const car1 = new Car("Tata", "Nexon");
const car2 = new Car("Mahindra", "Thar");
```

4. Kyu Use Karte Hain Classes aur Objects?

- Code repeat nahi hota
- Code organized aur maintain karna easy
- Real-world projects mein bahut useful
- Inheritance, Encapsulation jaise features milte hain

5. Properties Kya Hain? (Detailed Explanation)

Property = Object/Class ke andar stored data (variable). Yeh state ko represent karti hai.

Types of Properties:

Type	Syntax	Access	Example
Public	this.name	Bahar se bhi	this.name = "Pawan"
Protected	_name	Convention (warning)	_name
Private	#name	Sirf class ke andar	#balance

6. Properties Access Karne Ke Sab Tarike (Yeh Wala Section Pehle Miss Ho Gaya Tha)

Object Literal mein access karne ke sab tarike:

JavaScript

```
const student = {
    name: "Pawan",
    "full-name": "Pawan Kumar", // space/hyphen wali key
    marks: 94.4,
    city: "Haryana"
};
// 1. Dot Notation (sabse common)
console.log(student.name); // Pawan
console.log(student.marks); // 94.4
// 2. Bracket Notation (sabse powerful)
console.log(student["full-name"]); // Pawan Kumar
console.log(student["marks"]);
// 3. Dynamic Key (variable se)
let key = "city";
console.log(student[key]); // Haryana
// 4. Advanced Methods
console.log(Object.keys(student)); // ["name", "full-name", "marks", "city"]
console.log(Object.values(student)); // ["Pawan", "Pawan Kumar", 94.4, "Haryana"]
```

```
console.log(Object.entries(student)); // [["name","Pawan"], ...]
// 5. for...in loop (prototype chain bhi traverse karta hai)
for (let k in student) {
    console.log(k + " = " + student[k]);
}
// 6. hasOwnProperty() → sirf apni property check
console.log(student.hasOwnProperty("name")); // true
console.log(student.hasOwnProperty("toString")); // false (ye prototype se hai)
// 7. in operator (prototype chain bhi check karta hai)
console.log("name" in student); // true
console.log("toString" in student); // true
```

Class mein bhi same tarike kaam karte hain:

JavaScript

```
class Student {
    constructor(name) {
            .name = name;
            .#secret = "123"; // private
    }
    get fullName() {
        return     .name;
    }
}
const s = new Student("Pawan");
console.log(s.name); // dot
console.log(s["name"]); // bracket
console.log(s.fullName); // getter
// s.#secret → Error (private hai)
```

Fayda:

- Dot → simple keys ke liye
- Bracket → dynamic keys, space, hyphen ke liye
- Object.keys/values → jab saari properties list karni ho

7. Getters & Setters (Bahut Important)

Getter → Property padhne (read) ke liye **Setter** → Property set (write) karne ke liye

Code:

JavaScript

```
class Student {
    constructor(name) {
            ._name = name;
    }
    get name() { // Getter
        return     ._name.toUpperCase();
    }
```

with real-world examples
Created by Pawan Soni
learn computer, control your future

```
    set name(newName) { // Setter
        if (newName.length > 2) {
                ._name = newName;
        } else {
            console.log("❌ Name too short!");
        }
    }
}
const s1 = new Student("Pawan");
console.log(s1.name); // PAWAN ← Getter
s1.name = "Rahul"; // Setter
console.log(s1.name); // RAHUL
```

Fayda: Validation + clean code.

8. new Keyword

new naya object banata hai aur constructor ko call karta hai. **Kaam:** Empty object → this bind → constructor run → object return.

9. Constructor

Class ke andar special method jo new ke saath automatically call hota hai.

JavaScript

```
class Car {
    constructor(brand, model) {
            .brand = brand;
            .model = model;
    }
}
```

10. _ (Underscore) Convention

- Protected property indicate karta hai
- Bahar se use kar sakte ho lekin convention hai ki mat karo
- Real private nahi hai

11. # Private Fields & Methods

se shuru → real private (bahar se access nahi ho sakta).

JavaScript

```
class BankAccount {
    #balance = 0;
    deposit(amount) {
            .#balance += amount;
    }
    getBalance() {
        return      .#balance;
    }
}
```

Created by Pawan Soni

12. Inheritance (extends + super)

JavaScript

```
class Person {
    constructor(name) {
            .name = name;
    }
    eat() { console.log("Eating..."); }
}
class Engineer extends Person {
    constructor(name, branch) {
        super(name); // zaroori
            .branch = branch;
    }
    work() {
        super.eat(); // parent method
        console.log("Solving problems");
    }
}
```

13. Static Methods & Properties

Class level.

JavaScript

```
class User {
    static totalUsers = 0;
    static getTotalUsers() { return User.totalUsers; }
}
```

14. Prototypal Inheritance

JavaScript mein classes nahi, **prototypes** se inheritance hota hai. Har object ka ek hidden __proto__ hota hai jo dusre object se link hota hai.

Real-life analogy: Ek template (prototype) se saari copies ban jaati hain aur changes template mein karo to saari copies mein reflect hota hai.

JavaScript

```
// Old way (ES5)
function Person(name) {
        .name = name;
}

// Method prototype pe add karo → memory efficient
Person.prototype.greet = function() {
    console.log(`Hello, I'm ${    .name}`);
};

const p1 = new Person("Pawan");
```

```
const p2 = new Person("Rahul");

p1.greet(); // Hello, I'm Pawan
p2.greet(); // Hello, I'm Rahul

console.log(p1.__proto__ === Person.prototype); // true
console.log(Object.getPrototypeOf(p1) === Person.prototype); // modern way
```

Important: ES6 class actually yahi prototype chain use karti hai (sirf syntactic sugar hai).

Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION

15. Constructor Functions (Pre-ES6 Style)

Pehle classes nahi the, log constructor functions + prototype use karte the.

JavaScript

```
function Car(brand, model) {
        .brand = brand;
        .model = model;
}

Car.prototype.start = function() {
    console.log(`${    .brand} ${    .model} started!`);
};

const c1 = new Car("Tata", "Nexon");
c1.start();
```

Fayda: Methods prototype pe hone se har object mein duplicate nahi banta → memory saving.

16. Method Overriding

Child class parent ki same naam ki method ko override (redefine) kar sakti hai.

JavaScript

```
class Animal {
    speak() {
        console.log("Some generic sound...");
    }
}

class Dog extends Animal {
    speak() {                    // Overriding
        console.log("Bark Bark 🐶");
    }
}

class Cat extends Animal {
    speak() {
        console.log("Meow Meow 🐱");
    }
}

const d = new Dog();
const c = new Cat();
d.speak(); // Bark Bark
c.speak(); // Meow Meow
```

THE ARTIFICIAL EDUCATION

17. Polymorphism (Ek naam, alag-alag behaviour)

Ek hi method different objects ke liye alag-alag kaam kar sake.

JavaScript

```
class Shape {
    area() {
        return "Calculating area...";
    }
}

class Circle extends Shape {
    constructor(r) { super();     .radius = r; }
    area() { return Math.PI *     .radius ** 2; }
}

class Rectangle extends Shape {
    constructor(l, b) { super();     .length = l;     .breadth = b; }
    area() { return     .length *     .breadth; }
}

function printArea(shape) {
    console.log(shape.area());
}

printArea(new Circle(5));      // ~78.54
printArea(new Rectangle(4, 6)); // 24
```

Polymorphism = same method name, different implementation.

18. Abstraction Sirf zaruri details

expose karo, implementation hide karo.

JS mein native abstract class nahi hai, lekin hum private fields/methods + getters-setters se achieve karte hain.

JavaScript

Created by Pawan Soni

```
class Database {
    #connection; // private

    constructor() {
            .#connect(); // internal
    }

    #connect() { // private method
        console.log("Connecting to DB...");
            .#connection = "Connected";
```

```
    }

    query(sql) {
        // user ko sirf yeh method chahiye
        return `Executing: ${sql}`;
    }
}

const db = new Database();
db.query("SELECT * FROM users");
// db.#connect() → Error (hidden hai)
```

19. 'this' Keyword – Pura Scene (Bahut Important)

Situation	this kya point karta hai
Normal method (class)	jis object se call kiya gaya
Arrow function	lexical (parent scope ka this)
Regular function	call site pe depend
(loose)	(window/undefined)
bind(), call(), apply()	manually set kar sakte ho

JavaScript

```
class Test {
    constructor() {
            .name = "Pawan";
    }

    normalMethod() {
        console.log(    .name); // "Pawan"
    }

    arrowMethod = () => {
        console.log(    .name); // "Pawan" (lexical)
    }
}
```

Tip: Class ke andar arrow function use karo agar this binding issue ho.

20. instanceof Operator + Extra Checks

JavaScript

```
const car1 = new Car("Tata", "Nexon");

console.log(car1 instanceof Car);        // true
console.log(car1 instanceof Object);     // true
console.log(car1 instanceof Array);      // false
```

```
// Prototype check
console.log(Car.prototype.isPrototypeOf(car1)); // true
```

Bonus Best Practice (Interview mein puchha jaata hai):

Composition over Inheritance Inheritance se zyada **composition** better hota hai (has-a relationship).

JavaScript

```
class Engine {
    start() { console.log("Engine started"); }
}

class Car {
    constructor() {
            .engine = new Engine(); // composition
    }
    start() {
            .engine.start();
    }
}
```

Practical Questions

1. Question

Create a Car class with brand, model, and year. Add a start() method. Create 2 objects and call start().

Answer:

JavaScript

```
class Car {
    constructor(brand, model, year) {
            .brand = brand;
            .model = model;
            .year = year;
    }

    start() {
        console.log(`${    .brand} ${    .model} (${    .year}) started! 🚗`);
    }
}

const c1 = new Car("Tata", "Nexon", 2024);
const c2 = new Car("Mahindra", "Thar", 2025);
c1.start();
c2.start();
```

2. Question

THE ARTIFICIAL EDUCATION

What is the difference between Object Literal and Class? Give one example of each.

Answer: Object Literal → one-time object **Class** → blueprint for multiple objects

JavaScript

```
// Object Literal
const phone = { brand: "OnePlus", model: "12" };

// Class
class Phone {
    constructor(brand, model) {     .brand = brand;     .model = model; }
}
```

3. Question

Explain step-by-step what happens when we write new Car("Tata", "Nexon") (new keyword ka full working).

Answer:

1. Empty object {} banata hai
2. this ko us object se bind karta hai
3. Constructor run karta hai
4. Prototype link set karta hai
5. Final object return karta hai

4. Question

Create a class BankAccount with private #balance. Add deposit() and getBalance() methods.

Answer:

JavaScript

```
class BankAccount {
    #balance = 0;

    deposit(amount) {
        if (amount > 0)     .#balance += amount;
    }

    getBalance() {
        return     .#balance;
    }
}

const acc = new BankAccount();
acc.deposit(5000);
console.log(acc.getBalance()); // 5000
// console.log(acc.#balance); → Error (private hai)
```

5. Question

What will be the output?

JavaScript

```
class Student {
    constructor(name) {
            .name = name;
    }
    get name() {
        return     ._name.toUpperCase();
    }
    set name(newName) {
            ._name = newName;
    }
}

const s = new Student("pawan");
s.name = "rahul";
console.log(s.name);
```

Answer: RAHUL (Getter automatically call hota hai)

6. Question

Create two classes Person and Engineer using Inheritance. Engineer should have extra branch and override greet() method.

Answer:

JavaScript

```
class Person {
    constructor(name) {     .name = name; }
    greet() { console.log(`Hi, I'm ${    .name}`); }
}

class Engineer extends Person {
    constructor(name, branch) {
        super(name);
            .branch = branch;
    }
    greet() {                                // Overriding
        super.greet();
        console.log(`I am ${    .branch} Engineer`);
    }
}
```

```
const e = new Engineer("Pawan", "CSE");
e.greet();
```

7. Question

What is super() and why is it mandatory in child constructor?

Answer: super() parent class ka constructor call karta hai. Bina super() ke child constructor mein this use nahi kar sakte → Error.

8. Question

Create a class MathUtils with static method add() and static property PI.

Answer:

JavaScript

```
class MathUtils {
    static PI = 3.1416;

    static add(a, b) {
        return a + b;
    }
}

console.log(MathUtils.add(5, 3));     // 8
console.log(MathUtils.PI);            // 3.1416
```

9. Question

What will happen if we call method without new?

JavaScript

```
function Test() {
        .name = "Pawan";
}
Test();                    // without new
console.log(name);         // ???
```

Answer: name global variable ban jayega (window.name in browser). **Solution:** Hamesha new Test() use karo.

10. Question

Explain this keyword in 4 different cases with example.

Answer:

Case	this points to	Example
Normal method	Current object	obj.method()
Arrow function	Lexical (parent)	() => this

JavaScript Complete Notes
Learn JavaScript from basics to advanced with real-world examples
Prepared by Pawan Sahu
learn computer, control your future
THE ARTIFICIAL EDUCATION

Regular function	window / undefined (strict)	function(){}
call() / bind()	Manually set	func.call(obj)

11. Question

What is method overriding? Show with Animal → Dog example.

Answer: (Question 6 mein already diya hai – Dog class speak() override karti hai)

12. Question

Create a Shape parent class and Circle, Rectangle child classes (Polymorphism example).

Answer:

JavaScript

```
class Shape {
    area() { return 0; }
}

class Circle extends Shape {
    constructor(r) { super();     .r = r; }
    area() { return Math.PI *     .r *     .r; }
}

class Rectangle extends Shape {
    constructor(l, b) { super();     .l = l;     .b = b; }
    area() { return     .l *     .b; }
}

const shapes = [new Circle(5), new Rectangle(4, 6)];
shapes.forEach(s => console.log(s.area()));
```

13. Question

Difference between _name and #name?

Answer: _name = Protected (convention only) → bahar se access ho sakta hai #name = Real Private → bahar se access nahi ho sakta (Error)

14. Question

How to check if an object is instance of a class? (instanceof)

Answer:

JavaScript

```
const c = new Car("Tata", "Nexon");
console.log(c instanceof Car);        // true
console.log(c instanceof Object);     // true
```

15. Question

What is Prototypal Inheritance? Show old way (constructor function).

Answer:

JavaScript

```
function Person(name) {      .name = name; }
Person.prototype.greet = function() {
    console.log("Hello " +      .name);
};
const p = new Person("Pawan");
p.greet();
```

16. Question

Create a class with dynamic property using bracket notation.

Answer:

JavaScript

```
class Student {
    constructor() {
            ["full-name"] = "Pawan Kumar";
    }
}
const s = new Student();
console.log(s["full-name"]);
```

17. Question

What is Abstraction? Kaise implement karte hain JS mein?

Answer: Sirf necessary details expose karo. Private fields + methods use karke.

JavaScript

```
class Database {
    #connection;
    constructor() {      .#connect(); }
    #connect() { console.log("DB connected"); }
    query() { return "Data fetched"; }
}
```

18. Question

Write code to use Object.keys(), Object.values() and for...in on a class object.

Answer:

JavaScript

```
const car = new Car("Tata", "Nexon");
console.log(Object.keys(car));
console.log(Object.values(car));
```

```
for (let key in car) {
    console.log(key, car[key]);
}
```

19. Question

Composition vs Inheritance – Ek simple example do.

Answer:

JavaScript

```
class Engine { start() { console.log("Engine on"); } }

class Car {                    // Composition (better)
    constructor() {     .engine = new Engine(); }
    start() {     .engine.start(); }
}
```

20. Question (Advanced)

Create a User class with private #password. Add getter/setter with validation (password length > 6).

Answer:

JavaScript

```
class User {
    #password;

    set password(pass) {
        if (pass.length > 6)     .#password = pass;
        else console.log("❌ Password too short!");
    }

    get password() {
        return "******";   // hide real password
    }
}

const u = new User();
u.password = "1234567";
console.log(u.password);   // ******
```

✅ Asynchronous JavaScript

1. Synchronous vs Asynchronous Programming

Synchronous Programming

- Code line-by-line, sequence mein chalta hai.
- Ek instruction complete hone tak agla wait karta hai.

- Jaise real life mein ek kaam khatam kiye bina dusra nahi shuru karte.

Example:

JavaScript

```
console.log("1");
console.log("2");
console.log("3");
console.log("4");
```

Output: 1 → 2 → 3 → 4 (ek ke baad dusra)

Asynchronous Programming

- Kuch tasks jo time lete hain (jaise API call, setTimeout) unko background mein chala do.
- Baaki code wait nahi karta – parallel mein chalta hai.
- Jab heavy task complete ho jaye tab uska result aa jaye.

Real-life analogy: Amazon order kar diya → wait nahi karte, apna kaam karte rehte hain. Jab order deliver ho jaye tab use kar lenge.

JavaScript mein Asynchronous ka sabse simple example – setTimeout

JavaScript

```
console.log("1");
console.log("2");

setTimeout(() => {
    console.log("Hello after 2 seconds! 🚀");
}, 2000);

console.log("3");
console.log("4");
```

Output: 1 2 3 4 (2 seconds baad) Hello after 2 seconds!

2. Callback Kya Hai?

Definition: Ek function ko dusre function ke andar as an **argument** pass karna. Jab andar wala function call hota hai tab usko **callback** kehte hain.

Basic Example:

JavaScript

```
function calculator(a, b, sumCallback) {
    sumCallback(a, b);
}

function sum(a, b) {
    console.log(a + b);
}

calculator(5, 10, sum);        // 15
```

Arrow function se bhi pass kar sakte ho:

JavaScript

```
calculator(5, 10, (a, b) => {
    console.log(a + b);
});
```

setTimeout mein bhi callback:

JavaScript

```
setTimeout(() => {
    console.log("Hello");
}, 2000);
```

Important Rule: Callback ko **without parentheses** pass karo → sum (na ki sum())

3. Callback Hell (Pyramid of Doom) 😱

Jab callbacks ek dusre ke andar nested ho jayein (bahut gehraai tak) to code **Callback Hell** ban jata hai. Code samajhna mushkil ho jata hai, maintain karna almost impossible.

Real-life example (getData from database):

JavaScript

```
getData(1, () => {                    // 2 sec delay
    getData(2, () => {                // 2 sec delay
        getData(3, () => {            // 2 sec delay
            getData(4, () => {
                console.log("Done");
            });
        });
    });
});
```

Problem:

- Code pyramid jaisa ban jata hai.
- Bug fix karna bahut mushkil.
- New developer ko samajh nahi aata.
- Isliye isko **Callback Hell** ya **Pyramid of Doom** kehte hain.

4. Promises – Callback Hell ka Solution

Promise ek object hota hai jo future mein complete hone wale task ka **promise** (vada) karta hai.

3 States of Promise:

State	Matlab	Result
Pending	Kaam chal raha hai	undefined
Fulfilled	Kaam successfully complete	value
Rejected	Error aaya	error object

Complete Notes
with real-world examples
Created by Pawan Soni
learn computer, control your future
THE ARTIFICIAL EDUCATION

Promise create karna:

JavaScript

```
const promise = new Promise((resolve, reject) => {
    // heavy task
    let success = true;

    if (success) {
        resolve("Data fetched successfully! ✅");   // fulfilled
    } else {
        reject("Network error ❌");                 // rejected
    }
});
```

Promise use karna (.then & .catch):

JavaScript

```
promise
    .then((result) => {
        console.log(result);      // success
    })
    .catch((error) => {
        console.log(error);       // error
    });
```

5. Promise Chaining

Ek promise ke baad dusra promise call karna (without nesting).

Example (sequential data fetching):

JavaScript

```
getData(1)
    .then((data) => {
        console.log("Data 1:", data);
        return getData(2);            // next promise
    })
    .then((data) => {
        console.log("Data 2:", data);
        return getData(3);
    })
    .then((data) => {
        console.log("Data 3:", data);
    })
    .catch((err) => console.log(err));
```

Fayda: Code flat rehta hai → Callback Hell nahi hota.

6. Async / Await (Sabse Best & Modern Way) 🔥

- async → function ko asynchronous bana deta hai (hamesha Promise return karta hai).
- await → promise ke resolve hone tak wait karta hai (sirf async function ke andar use kar sakte ho).

Syntax:

JavaScript

```
async function getAllData() {
    console.log("Getting Data 1...");
    let data1 = await getData(1);      // wait karega
    console.log("Data 1 received");

    console.log("Getting Data 2...");
    let data2 = await getData(2);
    console.log("Data 2 received");
}

getAllData();
```

Super clean & readable!

7. IIFE (Immediately Invoked Function Expression) + Async/Await

Agar top-level pe directly await use karna ho (function ke bahar) to IIFE use karte hain.

JavaScript

```
(async () => {
    let data1 = await getData(1);
    let data2 = await getData(2);
    console.log("Done");
})();
```

8. Final Conclusion (Lecture ka end message)

Method	Readability	Maintenance	Real Projects mein use
Callbacks	Bahut kam	Bahut mushkil	Sirf simple cases
Promises + .then	Medium	Achha	Common
Async/Await	**Sabse best**	**Sabse easy**	**Sabse zyada use**

Questions with Answers

1. Question

setTimeout ke saath ek simple callback banao jo 3 seconds baad "Hello World" print kare.

Answer:

JavaScript

Created by Pawan Soni
learn computer control your future
THE ARTIFICIAL EDUCATION

```
setTimeout(() => {
    console.log("Hello World 🌍");
}, 3000);

console.log("Main code continues...");
```

Output: Main code continues... (3 sec baad) Hello World 🌍

Explanation: setTimeout mein arrow function callback hai.

2. Question

Ek calculator function banao jo 2 numbers aur ek callback le, aur callback mein unka sum print kare.

Answer:

JavaScript

```
function calculator(a, b, callback) {
    callback(a, b);
}

calculator(10, 20, (x, y) => {
    console.log("Sum =", x + y);   // 30
});
```

3. Question

Callback Hell ka example dikhao – 3 baar getData call karo (2 sec delay ke saath).

Answer:

JavaScript

```
function getData(id, callback) {
    setTimeout(() => {
        console.log(`Data ${id} fetched`);
        callback();
    }, 2000);
}

getData(1, () => {
    getData(2, () => {
        getData(3, () => {
            console.log("All data fetched ✅");
        });
    });
});
```

Yeh hi Callback Hell hai – pyramid jaisa code.

4. Question

Ek Promise banao jo 2 seconds baad "Data fetched successfully" resolve kare.

Answer:

JavaScript

```
const myPromise = new Promise((resolve, reject) => {
    setTimeout(() => {
        resolve("Data fetched successfully ✅");
    }, 2000);
});

myPromise.then((result) => {
    console.log(result);
});
```

5. Question

Ek Promise banao jo 50% chance se resolve ya reject ho. .then aur .catch dono use karo.

Answer:

JavaScript

```
const randomPromise = new Promise((resolve, reject) => {
    setTimeout(() => {
        if (Math.random() > 0.5) {
            resolve("Success! 🎉");
        } else {
            reject(new Error("Failed ❌"));
        }
    }, 1500);
});

randomPromise
    .then((res) => console.log(res))
    .catch((err) => console.log(err.message));
```

6. Question

Promise chaining karo – pehle getData(1) call karo, phir uske result ke baad getData(2).

Answer:

JavaScript

```
function getData(id) {
    return new Promise((resolve) => {
        setTimeout(() => {
            resolve(`Data ${id} received`);
        }, 1000);
    });
}
```

```
getData(1)
    .then((data) => {
        console.log(data);
        return getData(2);
    })
    .then((data) => console.log(data));
```

7. Question

Async/Await se upar wali chaining ko rewrite karo (sabse clean way).

Answer:

JavaScript

```
async function fetchAllData() {
    const data1 = await getData(1);
    console.log(data1);

    const data2 = await getData(2);
    console.log(data2);
}

fetchAllData();
```

8. Question

Async function mein error handle karne ke liye try...catch use karo.

Answer:

JavaScript

```
async function fetchData() {
    try {
        const data = await getData(1);
        console.log("Success:", data);
    } catch (error) {
        console.log("Error occurred:", error.message);
    }
}

fetchData();
```

9. Question

Ek function banao jo Promise return kare aur reject bhi ho sake. Uske liye try...catch use karo.

Answer:

JavaScript

```
function fetchUser(id) {
    return new Promise((resolve, reject) => {
```

```
        setTimeout(() => {
            if (id === 1) resolve({ name: "Pawan" });
            else reject(new Error("User not found"));
        }, 1000);
    });
}

async function getUser() {
    try {
        const user = await fetchUser(1);
        console.log(user);
    } catch (err) {
        console.log(err.message);
    }
}

getUser();
```

10. Question

Promise.all() use karke 3 promises ko parallel mein run karo.

Answer:

JavaScript

```
const p1 = new Promise(r => setTimeout(() => r("A"), 1000));
const p2 = new Promise(r => setTimeout(() => r("B"), 2000));
const p3 = new Promise(r => setTimeout(() => r("C"), 500));

Promise.all([p1, p2, p3])
    .then(results => console.log(results));   // ["A", "B", "C"]
```

11. Question

Ek real-life jaisa login function banao jo username check kare (Promise use karke).

Answer:

JavaScript

```
function login(username) {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (username === "pawan") {
                resolve("Login successful ✅");
            } else {
                reject(new Error("Invalid username"));
            }
        }, 1500);
```

Created by Pawan Soni

```
    });
}

async function attemptLogin() {
    try {
        const msg = await login("pawan");
        console.log(msg);
    } catch (err) {
        console.log(err.message);
    }
}

attemptLogin();
```

12. Question

IIFE ke saath top-level await use karo (function ke bahar).

Answer:

JavaScript

```
(async () => {
    try {
        const data1 = await getData(1);
        const data2 = await getData(2);
        console.log("All done:", data1, data2);
    } catch (err) {
        console.log(err);
    }
})();
```

13. Question

Ek function banao jo 3 seconds baad resolve ho, aur usko async/await + try...catch se call karo.

Answer:

JavaScript

Created by Pawan Soni

```
function delay(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
}

async function demo() {
    console.log("Starting...");
    try {
        await delay(3000);
        console.log("3 seconds passed ✅");
    } catch (e) {
```

```
        console.log("Error");
    }
}

demo();
```

14. Question

Complete Notes

.then().catch() aur async/await dono ko compare karte hue same kaam karo.

Answer (Both ways):

JavaScript

Learn JavaScript from basics to advanced

```
// Way 1: .then / .catch
getData(1)
    .then(d => { console.log(d); return getData(2); })
    .then(d => console.log(d))
    .catch(e => console.log(e));

// Way 2: async/await (Recommended)
async function fetchSeq() {
    try {
        const d1 = await getData(1);
        console.log(d1);
        const d2 = await getData(2);
        console.log(d2);
    } catch (e) {
        console.log(e);
    }
}
fetchSeq();
```

15. Question (Advanced)

Ek fetchWeather function banao jo Promise return kare. Agar city "Delhi" hai to resolve, warna reject. Phir async/await se call karo.

Created by Pawan Soni

Answer:

JavaScript

learn computer, control your future

```
function fetchWeather(city) {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (city.toLowerCase() === "delhi") {
                resolve({ temp: 32, condition: "Sunny ☀️" });
            } else {
                reject(new Error("City not found"));
            }
```

```
        }, 1200);
    });
}

async function getWeather() {
    try {
        const weather = await fetchWeather("Delhi");
        console.log("Weather:", weather);
    } catch (err) {
        console.log(err.message);
    }
}

getWeather();
```

✅ Fetch API

1. API Kya Hai? (Basic Revision)

API = Application Programming Interface Ek interface (bridge) jo do alag-alag software ko baat karne deta hai.

Real-life analogy: Restaurant mein waiter = API

- Tum order dete ho (Request)
- Waiter kitchen se khana laata hai (Response)
- Kitchen kaise banaya, ingredients kya the – yeh sab nahi pata chalta, sirf final dish milta hai.

Kaise kaam karta hai? Request → Server → Response (data) Data usually **JSON** format mein aata hai (JavaScript Object Notation).

Free APIs try karne ke liye:

- publicapis.dev
- Cat Facts API, Joke API, Weather API, etc.

Created by Pawan Soni

2. Fetch API Kya Hai?

JavaScript mein built-in method jo kisi URL (API endpoint) pe request bhejta hai aur data laata hai.

Syntax (Basic):

JavaScript

```
fetch("https://api.example.com/data")
```

Important points:

- fetch() hamesha **Promise** return karta hai.
- Pehle Promise pending hota hai → phir fulfilled (data aata hai) ya rejected (error).

Full Practical Example (Cat Facts API):

THE ARTIFICIAL EDUCATION

JavaScript

```
const BASE_URL = "https://catfact.ninja/fact";

async function getFact() {
    console.log("Getting data...");

    const response = await fetch(BASE_URL);   // Promise resolve → Response
object
    const data = await response.json();       // JSON → JavaScript object

    console.log(data);                        // Actual data
}

getFact();
```

3. Fetch se Milne Wala Response Object

fetch() se jo object milta hai usko **Response** kehte hain.

Important Properties:

- response.status → 200 (success), 404 (not found), 500 (server error)
- response.ok → true/false (200-299 range mein true)
- response.json() → JSON ko JavaScript object mein convert karta hai (yeh bhi Promise return karta hai)

Status Codes Quick Summary:

- 200 → Success (sab kuch theek)
- 404 → Not Found
- 400 → Bad Request
- 500 → Server Error

4. JSON vs XML (Data Format)

- **JSON** (JavaScript Object Notation) → Aaj kal sabse common. Object jaisa dikhta hai.
- **XML** → Purana format, ab kam use hota hai.

AJAX = Asynchronous JavaScript and XML (Aaj kal isko **AJAJ** bhi bolte hain kyunki JSON zyada use hota hai, lekin naam AJAX hi popular hai)

JSON convert karne ka tarika:

JavaScript

```
const data = await response.json();   // Yeh line JSON ko usable object bana deti
hai
```

5. Fetch API with Async/Await (Recommended Way)

JavaScript

Created by Pawan Soni
Own computer, control your future
THE ARTIFICIAL EDUCATION

```javascript
async function getData() {
    try {
        const response = await fetch(BASE_URL);

        if (!response.ok) {
            throw new Error("API call failed");
        }

        const data = await response.json();
        console.log(data);
    } catch (error) {
        console.log("Error:", error);
    }
}
```

Promise Chaining se bhi kar sakte ho (purana tarika):

JavaScript

```javascript
fetch(BASE_URL)
    .then(res => res.json())
    .then(data => console.log(data))
    .catch(err => console.log(err));
```

Conclusion: Async/Await zyada clean aur readable hai.

6. HTTP Methods (GET, POST, etc.)

Method	Kaam	Kab use karte hain?
GET	Data lene ke liye	Sirf padhne ke liye
POST	Data bhejne ke liye	Naya data create karne ke liye
PUT	Poora data update	Existing data replace
PATCH	Thoda data update	Partial update
DELETE	Data delete karne ke liye	Delete operation

Default fetch method = GET

Created by Pawan Soni

✅ LocalStorage in JavaScript

1. LocalStorage Kya Hai?

- Yeh **key-value** pair mein data save karta hai.
- Data **string** format mein save hota hai.
- Limit: ~5MB per domain (bahut zyada hai normal use ke liye).
- Tab band karne ya browser restart karne pe bhi data safe rahta hai.

SessionStorage se farak:

THE ARTIFICIAL EDUCATION

Feature	LocalStorage	SessionStorage
Data kitne time tak	Hamesha (permanent)	Sirf tab tak
Browser restart	Data rahta hai	Data delete ho jata hai
Use case	User settings, cart	Temporary form data

2. Important Methods (Sabse Zyada Use Hone Wale)

JavaScript

```javascript
// 1. Data save karna
localStorage.setItem('key', 'value');

// 2. Data nikaalna
localStorage.getItem('key');

// 3. Data delete karna
localStorage.removeItem('key');

// 4. Saara data clear karna
localStorage.clear();

// 5. Kitne items hain
localStorage.length;

// 6. Key ka naam index se nikaalna
localStorage.key(0);
```

3. Practical Examples (Copy-Paste Ready)

JavaScript

```javascript
// String save karna
localStorage.setItem('username', 'Pawan');

// Number save karna (string ban jaata hai)
localStorage.setItem('age', '25');

// Object save karna (Important!)
const user = { name: "Pawan", city: "Haryana", age: 25 };
localStorage.setItem('user', JSON.stringify(user));

// Object wapas nikaalna
const savedUser = JSON.parse(localStorage.getItem('user'));
console.log(savedUser.name);   // Pawan
```

JavaScript Complete Notes
THE ARTIFICIAL EDUCATION

10 Practical Questions

1. Ek button click karne pe username "Pawan" save karo LocalStorage mein.

Solution:

JavaScript

```
function saveUser() {
    localStorage.setItem('username', 'Pawan');
    console.log('✅ Username saved!');
}
```

2. Page load hone pe saved username ko console mein print karo.

Solution:

JavaScript

```
window.onload = function() {
    const name = localStorage.getItem('username');
    if (name) {
        console.log('Saved Username:', name);
    } else {
        console.log('No username found');
    }
};
```

3. Function banao jo name aur city save kare (object ke form mein).

Solution:

JavaScript

```
function saveProfile(name, city) {
    const profile = { name, city };
    localStorage.setItem('profile', JSON.stringify(profile));
    console.log('Profile saved');
}

saveProfile('Pawan', 'Haryana');
```

4. Saved profile object ko nikaal ke naam aur city alag-alag print karo.

Solution:

JavaScript

```
const profile = JSON.parse(localStorage.getItem('profile'));
console.log('Name:', profile.name);
console.log('City:', profile.city);
```

5. Ek specific key (age) ko delete karo.

Solution:

JavaScript

```
localStorage.removeItem('age');
console.log('Age deleted');
```

6. Saare LocalStorage data ko clear karo (logout feature jaise).

Solution:

JavaScript

```
function logout() {
    localStorage.clear();
    console.log('All data cleared - Logged out');
}
```

7. Current LocalStorage mein kitne items hain aur unki keys print karo.

Solution:

JavaScript

```
console.log('Total items:', localStorage.length);

for (let i = 0; i < localStorage.length; i++) {
    const key = localStorage.key(i);
    console.log(i + 1, '→ Key:', key, '| Value:', localStorage.getItem(key));
}
```

8. Dark mode preference save karo aur page load pe apply karo.

Solution:

JavaScript

```
// Save karne ke liye
function toggleDarkMode() {
    const isDark = document.body.classList.toggle('dark');
    localStorage.setItem('darkMode', isDark ? 'true' : 'false');
}

// Page load pe apply karne ke liye
if (localStorage.getItem('darkMode') === 'true') {
    document.body.classList.add('dark');
}
```

9. Shopping cart mein ek product add karo aur localStorage mein save karo (array ke form mein).

Solution:

JavaScript

```
let cart = JSON.parse(localStorage.getItem('cart')) || [];

function addToCart(product) {
    cart.push(product);
    localStorage.setItem('cart', JSON.stringify(cart));
    console.log('Added to cart');
}

addToCart({ id: 1, name: 'iPhone', price: 79900 });
```

10. Page load hone pe cart ko load karo aur total items ki count dikhao.

Solution:

Complete Notes
THE ARTIFICIAL EDUCATION

JavaScript

```javascript
window.onload = function() {
    const cart = JSON.parse(localStorage.getItem('cart')) || [];
    console.log('Items in Cart:', cart.length);

    // Agar HTML mein dikhana ho
    document.getElementById('cartCount').innerText = cart.length;
};
```


Created by **Pawan Soni**

learn computer, control your future

THE ARTIFICIAL EDUCATION